ÉCOLE MAROCAINE DES SCIENCES DE L'INGÉNIEUR

EMSI

PROJET DE FIN D'ANNÉE

RAPPORT DE PFA

THÈME DU PROJET

Système de Gestion Interne pour BMT Tourisme

Conception et développement d'une application web de gestion pour la centralisation et le suivi des activités d'une agence de voyage

RÉALISÉ PAR

ZAKARIA M'AYZOU

ENCADRÉ PAR

Mme Mariem El Hlili

ORGANISME D'ACCUEIL

BMT Tourisme

ÉTABLISSEMENT

EMSI

ANNÉE UNIVERSITAIRE

2025 – 2026

Dédicace

Je dédie ce travail,

À mes chers parents, pour leur amour, leur soutien, leurs sacrifices et leurs encouragements tout au long de mon parcours académique.

À ma famille, pour sa présence, sa confiance et son soutien constant.

À mes enseignants et à toutes les personnes qui ont contribué, de près ou de loin, à ma formation et à mon développement.

À mes amis et à tous ceux qui m'ont accompagné et encouragé durant la réalisation de ce projet.

Zakaria M'AYZOU

Remerciements

Au terme de ce Projet de Fin d'Année, je tiens à exprimer ma profonde gratitude à toutes les personnes qui ont contribué, de près ou de loin, à la réalisation de ce travail.

Je tiens tout particulièrement à adresser mes sincères remerciements à mon encadrante, **Mme Maryem El Hlili**, pour son accompagnement, sa disponibilité, ses conseils et ses orientations tout au long de la réalisation de ce projet. Son suivi m'a permis de mieux structurer mon travail et de progresser dans les différentes étapes de sa réalisation.

J'adresse également mes remerciements à **l'École Marocaine des Sciences de l'Ingénieur (EMSI)**, ainsi qu'à l'ensemble du corps professoral, pour la qualité de la formation dispensée et pour les connaissances acquises tout au long de mon parcours académique.

Je remercie également **BMT Tourisme** pour l'opportunité qui m'a été accordée de travailler sur un projet répondant à des besoins concrets de gestion interne d'une agence de voyage. Cette expérience m'a permis de mettre en pratique mes connaissances techniques et de mieux comprendre les exigences liées à la conception et au développement d'une application destinée à un environnement professionnel.

Enfin, je remercie ma famille, mes proches et toutes les personnes qui m'ont soutenu et encouragé durant la réalisation de ce projet.

M'AYZOU ZAKARIA

Résumé

Dans le cadre de ce Projet de Fin d'Année, une application web de gestion interne a été conçue et développée afin de répondre aux besoins organisationnels de **BMT Tourisme** et de centraliser les différentes activités liées à la gestion d'une agence de voyage.

La solution proposée permet de gérer les principales opérations de l'agence à travers une plateforme unique. Elle intègre notamment la gestion des clients et de leurs dossiers, le suivi des demandes de visa, la billetterie, les paiements et avances, l'organisation des tâches, ainsi que la gestion des guides et de leurs missions. Le système intègre également des fonctionnalités de notification, de suivi des actions et de consultation d'indicateurs permettant d'améliorer la visibilité sur l'activité de l'agence.

Une attention particulière a été accordée à la gestion des accès. L'application repose sur un système de rôles et de permissions permettant d'adapter les fonctionnalités disponibles selon le profil de chaque employé. La gestion des agences permet également de limiter l'accès aux données selon l'agence d'affectation, tout en autorisant les profils disposant des droits nécessaires à consulter les données à un niveau global.

Sur le plan technique, l'application a été développée avec le framework **Django** en utilisant **Python** pour la partie serveur et **MySQL** pour la gestion de la base de données. L'interface utilisateur repose sur **HTML**, **CSS**, **Bootstrap** et **JavaScript**, afin de proposer une interface moderne, structurée et adaptée aux différents utilisateurs du système.

Mots-clés : BMT Tourisme, gestion d'agence de voyage, Django, Python, MySQL, gestion des clients, dossiers, visa, billetterie, paiements, tâches, rôles et permissions.

Abstract

As part of this End-of-Year Project, an internal web-based management application was designed and developed to address the organizational needs of **BMT Tourisme** and to centralize the main activities involved in the management of a travel agency.

The proposed solution provides a unified platform for managing the agency's main operations. It includes customer and file management, visa application tracking, ticketing, payments and advance payments, task organization, as well as the management of guides and their missions. The system also provides notification, action tracking, and reporting features to improve visibility and monitoring of the agency's activities.

Particular attention was given to access control. The application is based on a role and permission management system that adapts the available functionalities according to each employee's profile. Agency-based access control also restricts data according to the employee's assigned agency, while authorized profiles can access information at a global level.

From a technical perspective, the application was developed using the **Django** framework with **Python** for the server-side logic and **MySQL** for database management. The user interface is based on **HTML**, **CSS**, **Bootstrap**, and **JavaScript**, providing a modern, structured, and user-friendly environment adapted to the different users of the system.

Keywords : BMT Tourisme, travel agency management, Django, Python, MySQL, customer management, files, visa, ticketing, payments, tasks, roles and permissions.

Liste des abréviations

PFA	Projet de Fin d'Année
EMSI	École Marocaine des Sciences de l'Ingénieur
UML	Unified Modeling Language
MVT	Model – View – Template
HTML	HyperText Markup Language
CSS	Cascading Style Sheets
JS	JavaScript
SQL	Structured Query Language
SGBD	Système de Gestion de Base de Données
UI	User Interface
URL	Uniform Resource Locator
CRUD	Create, Read, Update, Delete
CSRF	Cross-Site Request Forgery

Table des matières

Table des matières ix

Liste des figures xviii

Liste des tableaux xix

Introduction Générale 1

1 Présentation de l'Organisme d'Accueil et du Projet 4

1.1 Introduction . . . 4
1.2 Présentation de BMT Tourisme . . . 4
1.3 Organisation générale . . . 4
1.4 Contexte du projet . . . 5
1.5 Présentation du projet . . . 6
1.6 Principe de fonctionnement général . . . 6
1.7 Gestion multi-agences . . . 7
1.8 Apports attendus de la solution . . . 7
1.9 Conclusion . . . 7

2 Étude de l'Existant et Analyse des Besoins 8

2.1 Introduction . . . 8
2.2 Étude de l'existant . . . 8
2.3 Limites identifiées . . . 9
2.3.1 Dispersion des informations . . . 9
2.3.2 Suivi des dossiers . . . 9
2.3.3 Coordination entre les services . . . 9
2.3.4 Gestion des responsabilités . . . 9
2.3.5 Gestion de plusieurs agences . . . 9
2.3.6 Suivi financier . . . 10
2.4 Identification des acteurs . . . 10

2.5 Analyse des besoins fonctionnels . . . 11
2.5.1 Authentification et gestion des accès . . . 11
2.5.2 Gestion des clients . . . 11
2.5.3 Gestion des dossiers . . . 11
2.5.4 Gestion documentaire . . . 11
2.5.5 Gestion des tâches . . . 12
2.5.6 Gestion des paiements . . . 12
2.5.7 Gestion des visas . . . 12
2.5.8 Gestion de la billetterie . . . 12
2.5.9 Gestion des guides et des missions . . . 12
2.5.10 Notifications et traçabilité . . . 13
2.5.11 Tableau de bord et rapports . . . 13
2.6 Règles de gestion principales . . . 13
2.7 Besoins non fonctionnels . . . 14
2.7.1 Sécurité . . . 14
2.7.2 Ergonomie . . . 14
2.7.3 Performance . . . 14
2.7.4 Fiabilité . . . 14
2.7.5 Évolutivité . . . 14
2.7.6 Maintenabilité . . . 14
2.8 Synthèse des besoins . . . 15
2.9 Conclusion . . . 15

3 Spécifications Fonctionnelles 16
3.1 Introduction . . . 16
3.2 Authentification et accès au système . . . 16
3.3 Gestion des employés, rôles et permissions . . . 16
3.4 Gestion des agences . . . 17
3.5 Gestion des clients . . . 17
3.6 Gestion des dossiers . . . 18
3.7 Gestion des avances et traitement des dossiers . . . 18
3.8 Gestion des documents . . . 19

3.9 Gestion des tâches . . . 19
3.10 Gestion des paiements . . . 20
3.11 Gestion des visas . . . 20
3.12 Gestion de la billetterie . . . 20
3.13 Gestion des guides et des missions . . . 21
3.14 Notifications . . . 21
3.15 Historique des actions . . . 21
3.16 Tableau de bord . . . 22
3.17 Rapports et indicateurs . . . 22
3.18 Synthèse du fonctionnement . . . 23
3.19 Conclusion . . . 23

4 Architecture du Système et Choix Techniques 24
4.1 Introduction . . . 24
4.2 Architecture générale de l'application . . . 24
4.3 Framework Django . . . 24
4.4 Architecture MVT . . . 25
4.4.1 Model . . . 25
4.4.2 View . . . 25
4.4.3 Template . . . 25
4.5 Organisation modulaire du projet . . . 26
4.6 Langage Python . . . 27
4.7 Base de données MySQL . . . 27
4.8 Technologies Front-End . . . 27
4.8.1 HTML . . . 28
4.8.2 CSS . . . 28
4.8.3 Bootstrap . . . 28
4.8.4 JavaScript . . . 28
4.9 Principe du tableau de bord partagé . . . 28
4.10 Gestion des rôles et permissions . . . 28
4.11 Gestion du périmètre des agences . . . 29
4.12 Environnement de développement . . . 29

4.13 Justification des choix techniques 30
4.14 Conclusion 30

5 Conception UML du Système 32
5.1 Introduction 32
5.2 Diagramme de cas d'utilisation 32
5.3 Diagramme de classes 34
5.4 Diagramme de séquence 35
5.5 Diagramme d'activité 36
5.6 Diagramme d'états 38
5.7 Complémentarité des diagrammes 38
5.8 Conclusion 39

6 Conception et Gestion de la Base de Données 40
6.1 Introduction 40
6.2 Choix du système de gestion de base de données 40
6.3 Interaction entre Django et MySQL 41
6.4 Modélisation des données 41
6.5 Diagramme de classes 42
6.6 Description des principales entités 43
6.7 Relations entre les entités 45
6.7.1 Relation entre utilisateur et employé 45
6.7.2 Relation entre agence et employé 45
6.7.3 Relation entre rôle et permissions 45
6.7.4 Relation entre client et agence 45
6.7.5 Relation entre client et dossier 46
6.7.6 Relation entre dossier et agence 46
6.7.7 Relations autour du dossier 46
6.8 Le dossier comme entité centrale 46
6.9 Gestion des documents 47
6.10 Gestion des tâches 47
6.11 Gestion des données financières 48

6.11.1 Montant payé 48
6.11.2 Reste à payer 49
6.11.3 Gestion des avances 49
6.12 Gestion des visas 49
6.13 Gestion de la billetterie 49
6.14 Gestion des guides et des missions 50
6.15 Notifications et historique 50
6.15.1 Notification 51
6.15.2 HistoriqueAction 51
6.16 Contraintes et intégrité des données 51
6.16.1 Contraintes d'unicité 51
6.16.2 Relations obligatoires 52
6.16.3 Gestion des suppressions 52
6.17 Migrations Django 52
6.18 Gestion du périmètre multi-agences 53
6.19 Cohérence entre base de données et logique métier 53
6.20 Avantages de la conception adoptée 53
6.21 Conclusion 54

7 Réalisation et Description des Modules 55

7.1 Introduction 55
7.2 Interface d'authentification 56
7.3 Interface générale de l'application 57
7.4 Tableau de bord 58
7.5 Module de gestion des clients 59
7.5.1 Liste des clients 59
7.5.2 Ajout et modification d'un client 59
7.6 Module de gestion des dossiers 61
7.6.1 Liste des dossiers 61
7.6.2 Création d'un dossier 61
7.6.3 Détail d'un dossier 62
7.7 Module de gestion des tâches 63

7.8 Module de gestion des paiements 64
7.9 Module Visa 65
7.10 Module Billetterie 66
7.11 Module Guides et Missions 67
7.12 Module de gestion des employés 68
7.13 Module de gestion des agences 69
7.14 Notifications 70
7.15 Historique des actions 71
7.16 Rapports et indicateurs 72
7.17 Cohérence de l'interface utilisateur 73
7.18 Conclusion 74

8 Sécurité et Gestion des Accès 75

8.1 Introduction 75
8.2 Authentification des utilisateurs 75
8.3 Protection des pages authentifiées 75
8.4 Gestion des rôles 76
8.5 Permissions métier 76
8.6 Permissions spécifiques aux employés 77
8.7 Contrôle des permissions côté serveur 77
8.8 Décorateur de permissions métier 78
8.9 Gestion des utilisateurs inactifs 78
8.10 Contrôle d'accès multi-agences 78
8.11 Accès global et accès local 79
8.12 Filtrage des données côté serveur 79
8.13 Protection contre les accès directs 80
8.14 Validation des données envoyées 80
8.15 Protection des opérations sensibles 80
8.16 Protection CSRF 81
8.17 Sécurité au niveau de l'interface 81
8.18 Traçabilité des actions 81
8.19 Séparation entre interface et sécurité réelle 82

8.20 Synthèse de la stratégie de contrôle d'accès 82
8.21 Conclusion 83

9 Tests et Validation 84
9.1 Introduction 84
9.2 Objectifs des tests 84
9.3 Méthodologie de validation 85
9.4 Test de l'authentification 85
9.5 Test des rôles et permissions 85
9.6 Test du contrôle multi-agences 86
9.7 Test du module Clients 87
9.8 Test du module Dossiers 88
9.9 Test de la règle d'avance 88
9.10 Test du module Paiements 89
9.11 Test du module Tâches 90
9.12 Test des modules Visa, Billetterie et Missions 90
9.13 Test des notifications et de l'historique 91
9.14 Test des rapports 91
9.15 Validation de l'interface utilisateur 91
9.16 Bilan de validation 92
9.17 Conclusion 92

10 Difficultés Rencontrées et Solutions Apportées 93
10.1 Introduction 93
10.2 Structuration d'une application comportant plusieurs modules 93
10.2.1 Solution apportée 93
10.3 Gestion flexible des rôles et permissions 94
10.3.1 Solution apportée 94
10.4 Protection réelle des fonctionnalités 95
10.4.1 Solution apportée 95
10.5 Gestion du fonctionnement multi-agences 95
10.5.1 Solution apportée 95

10.6 Sécurisation des accès directs 96
10.6.1 Solution apportée 96
10.7 Gestion des permissions des guides 96
10.7.1 Solution apportée 96
10.8 Gestion de la règle métier liée à l'avance 97
10.8.1 Solution apportée 97
10.9 Calcul des paiements et remboursements 97
10.9.1 Solution apportée 97
10.10 Évolution des modèles et migrations 98
10.10.1 Solution apportée 98
10.11 Connexion à la base de données pendant le développement 98
10.11.1 Solution apportée 99
10.12 Cohérence entre les différents modules 99
10.12.1 Solution apportée 99
10.13 Amélioration de l'interface utilisateur 99
10.13.1 Solution apportée 99
10.14 Synthèse des difficultés 100
10.15 Apports de la résolution de ces difficultés 102
10.16 Conclusion 102

11 Perspectives d'Amélioration 103

11.1 Introduction 103
11.2 Déploiement sur un serveur de production 103
11.3 Amélioration du système de notifications 104
11.4 Gestion avancée des documents 104
11.5 Automatisation des tâches 104
11.6 Gestion avancée des échéances 105
11.7 Tableaux de bord plus avancés 105
11.8 Exportation des rapports 106
11.9 Amélioration de la gestion financière 106
11.10 Journalisation plus avancée 106
11.11 Renforcement de la sécurité 107

11.12Amélioration de l'expérience utilisateur 107
11.13Extension du fonctionnement multi-agences 108
11.14Intégration avec des services externes 108
11.15Application mobile 108
11.16Évolution vers une API 108
11.17Synthèse des perspectives 109
11.18Conclusion 110

Conclusion Générale **111**

Bibliographie et Webographie **113**

Annexes **115**

Annexe A – Organisation générale du projet 115
Annexe B – Principales entités du système 116
Annexe C – Permissions métier 118
Annexe D – États des principaux éléments 119
Annexe E – Principe du contrôle d'accès 120
Annexe F – Principe de la règle d'avance 121
Annexe G – Technologies principales 122
Annexe H – Interfaces complémentaires 122
Annexe I – Informations générales du projet 123

Table des figures

5.1 Diagramme de cas d'utilisation du système 33
5.2 Diagramme de classes du système 34
5.3 Diagramme de séquence du système 35
5.4 Diagramme d'activité du système 37
5.5 Diagramme d'états du système 38

6.1 Diagramme de classes du système de gestion interne 42

7.1 Interface d'authentification 56
7.2 Tableau de bord de l'application 58
7.3 Interface de gestion des clients 59
7.4 Formulaire de gestion d'un client 60
7.5 Liste des dossiers 61
7.6 Détail d'un dossier 62
7.7 Interface de gestion des tâches 63
7.8 Gestion des paiements 64
7.9 Interface de suivi des demandes de visa 65
7.10 Interface de gestion de la billetterie 66
7.11 Gestion des missions des guides 67
7.12 Interface de gestion des employés 68
7.13 Interface des notifications 70
7.14 Interface des rapports et indicateurs 72

Liste des tableaux

2.1 Principaux acteurs du système 10

4.1 Organisation des applications Django du projet 26
4.2 Principales technologies utilisées 30

6.1 Description des principales entités du système 44

8.1 Principe de gestion du périmètre des agences 79

9.1 Scénarios de test de l'authentification 85
9.2 Scénarios de test des rôles et permissions 86
9.3 Scénarios de test du contrôle multi-agences 87
9.4 Scénarios de test du module Clients 87
9.5 Scénarios de test du module Dossiers 88
9.6 Scénarios de validation de la règle d'avance 89
9.7 Scénarios de test des paiements 89
9.8 Scénarios de test du module Tâches 90
9.9 Tests des modules spécialisés 90

10.1 Synthèse des difficultés et solutions apportées 101

11.1 Synthèse des perspectives d'amélioration 109
2 Principales entités de l'application 117
3 Principales permissions métier 119
4 Technologies principales du projet 122
5 Informations générales du projet 123

Introduction Générale

Contexte général

La transformation numérique occupe aujourd'hui une place importante dans l'organisation et le fonctionnement des entreprises. Elle permet notamment de centraliser l'information, d'améliorer la communication entre les différents services et d'assurer un meilleur suivi des activités internes.

Dans le secteur du tourisme, les agences de voyage sont amenées à gérer quotidiennement un ensemble important d'informations liées aux clients, aux dossiers de voyage, aux demandes de visa, aux réservations, aux paiements, aux tâches internes ainsi qu'aux différentes prestations proposées. Lorsque ces informations sont réparties entre plusieurs supports ou traitées de manière indépendante par les différents services, leur suivi peut devenir complexe et entraîner une perte de temps dans les opérations quotidiennes.

C'est dans ce contexte que s'inscrit ce Projet de Fin d'Année, consacré à la conception et au développement d'un système de gestion interne destiné à **BMT Tourisme**.

Problématique

La gestion d'une agence de voyage implique l'intervention de plusieurs profils professionnels, notamment les commerciaux, les responsables visa, les agents de billetterie, les comptables, les guides et les responsables administratifs.

Chaque intervenant manipule des informations spécifiques tout en ayant besoin d'accéder à certaines données communes concernant les clients et leurs dossiers. Une gestion insuffisamment centralisée peut ainsi rendre plus difficile le partage de l'information, le suivi de l'avancement des dossiers, l'attribution des responsabilités et le contrôle des opérations effectuées.

La problématique principale de ce projet peut donc être formulée comme suit :

Comment concevoir une application web centralisée permettant d'organiser et de suivre efficacement les activités internes de BMT Tourisme, tout en garantissant une gestion adaptée des rôles, des permissions et des accès aux données ?

Solution proposée

Afin de répondre à cette problématique, le projet consiste à mettre en place une application web de gestion interne regroupant les principales opérations de l'agence au sein d'un environnement unique.

La solution permet de centraliser les informations relatives aux clients et à leurs dossiers, tout en facilitant le suivi des différentes opérations réalisées par les services concernés. Elle prend également en considération la structure organisationnelle de l'entreprise grâce à un mécanisme de rôles et de permissions permettant d'adapter les fonctionnalités accessibles à chaque utilisateur.

L'application vise ainsi à offrir une meilleure visibilité sur l'activité interne, à faciliter la circulation de l'information et à assurer un suivi plus structuré des opérations.

Objectifs du projet

L'objectif principal de ce projet est de concevoir et développer une solution informatique fiable, structurée et évolutive répondant aux besoins de gestion interne de BMT Tourisme.

Plus précisément, le projet vise à :

— centraliser les informations liées aux activités de l'agence ;
— faciliter la gestion et le suivi des clients et de leurs dossiers ;
— améliorer la coordination entre les différents services ;
— assurer le suivi des tâches et des opérations associées aux dossiers ;
— permettre une gestion structurée des paiements et des avances ;
— faciliter le suivi des opérations de visa et de billetterie ;
— organiser la gestion des guides et de leurs missions ;
— mettre en place une gestion des utilisateurs basée sur les rôles et les permissions ;
— assurer un contrôle cohérent de l'accès aux données des différentes agences ;
— fournir des indicateurs permettant de mieux suivre l'activité globale.

Démarche adoptée

La réalisation du projet a été organisée en plusieurs étapes complémentaires.

Une première phase a été consacrée à l'analyse du contexte et à l'identification des besoins fonctionnels. Cette étape a permis de déterminer les principaux acteurs du sys-

tème ainsi que les opérations qu'ils doivent pouvoir effectuer.

Une phase de conception a ensuite permis de structurer la solution, de définir les principales entités du système et de représenter son fonctionnement à travers différents diagrammes UML.

La phase de réalisation a consisté à développer progressivement les différents modules de l'application, à mettre en place la base de données et à intégrer les mécanismes d'authentification, d'autorisation et de contrôle d'accès.

Enfin, une phase de validation permettra de vérifier le comportement des fonctionnalités réalisées et leur conformité avec les besoins identifiés.

Organisation du rapport

Le présent rapport est structuré de manière à présenter progressivement les différentes étapes de réalisation du projet.

Il commence par une présentation de l'organisme d'accueil et du contexte du projet. Une étude de l'existant et une analyse des besoins permettent ensuite d'identifier les problématiques rencontrées et les exigences du futur système.

Les spécifications fonctionnelles sont ensuite présentées avant d'aborder l'architecture de la solution et les choix technologiques. La conception du système est détaillée à travers les différents diagrammes UML et la conception de la base de données.

Les chapitres suivants sont consacrés à la réalisation de l'application, à la présentation de ses principaux modules, à la sécurité et à la gestion des accès. Enfin, le rapport présente la démarche de validation, les difficultés rencontrées, les améliorations envisagées ainsi qu'une conclusion générale résumant les principaux résultats du projet.

Chapitre 1

Présentation de l'Organisme d'Accueil et du Projet

1.1 Introduction

Ce chapitre présente le contexte professionnel dans lequel s'inscrit le Projet de Fin d'Année. Il permet de situer l'organisme d'accueil, de comprendre son domaine d'activité et son organisation générale, puis de présenter le contexte ayant conduit à la réalisation du système de gestion interne proposé.

L'objectif est de fournir une vision globale de l'environnement du projet avant d'aborder, dans les chapitres suivants, l'analyse détaillée des besoins et la conception de la solution.

1.2 Présentation de BMT Tourisme

BMT Tourisme est une agence opérant dans le secteur du tourisme et du voyage. Son activité implique la gestion de plusieurs types de prestations ainsi que le traitement d'informations relatives aux clients, aux dossiers de voyage et aux différents services associés.

L'organisation de ces activités nécessite l'intervention de plusieurs profils professionnels spécialisés. Chaque service prend en charge une partie du traitement des demandes tout en collaborant avec les autres services afin d'assurer le suivi complet du dossier du client.

Dans ce contexte, la circulation de l'information et la coordination entre les différents intervenants représentent des éléments importants pour assurer une gestion structurée et efficace des opérations.

1.3 Organisation générale

BMT Tourisme dispose d'une organisation répartie entre plusieurs agences. Dans le cadre du présent projet, la structure prise en considération comprend une agence

principale située à **Casablanca**, ainsi que des agences à **Marrakech** et **Agadir**.

La solution développée a toutefois été pensée de manière évolutive. Elle ne limite pas le système à ces trois implantations et permet d'intégrer d'autres agences selon les futurs besoins de l'entreprise.

L'organisation fonctionnelle considérée dans l'application repose notamment sur les profils suivants :

— **Administrateur :** assure l'administration générale du système et la gestion des accès ;
— **Directeur :** dispose d'une vision globale sur l'activité et les différentes agences ;
— **Commercial :** assure principalement la gestion des clients et la création de leurs dossiers ;
— **Responsable Visa :** prend en charge le suivi des procédures et demandes liées aux visas ;
— **Agent Billetterie :** assure la gestion des réservations et des billets ;
— **Comptable :** assure le suivi des avances, des paiements et des opérations financières associées aux dossiers ;
— **Guide :** intervient dans les missions qui lui sont attribuées dans le cadre des activités de l'agence ;
— **Autres employés :** peuvent être intégrés au système selon leurs responsabilités et les permissions qui leur sont attribuées.

Cette organisation constitue une base importante pour la conception de l'application, puisque chaque utilisateur ne doit accéder qu'aux fonctionnalités et aux informations correspondant à ses responsabilités.

1.4 Contexte du projet

La gestion quotidienne d'une agence de voyage implique un enchaînement de plusieurs opérations. Une demande peut commencer par la prise en charge d'un client par un commercial, puis nécessiter l'intervention d'un ou plusieurs autres services selon la nature du dossier.

Un dossier peut, par exemple, nécessiter le traitement d'une demande de visa, une réservation de billet, l'enregistrement d'un paiement, l'affectation de tâches à différents employés ou encore l'organisation d'une mission impliquant un guide.

Ces opérations sont liées entre elles et nécessitent un suivi cohérent. La centralisation des données au sein d'un même système permet ainsi de disposer d'une vision plus claire

de l'état d'un dossier et des actions réalisées par les différents intervenants.

Le projet est donc né du besoin de disposer d'un outil interne capable de regrouper ces différentes opérations et d'améliorer leur suivi.

1.5 Présentation du projet

Le projet consiste à concevoir et développer une application web intitulée :

Système de Gestion Interne pour BMT Tourisme

Cette application constitue un espace de travail centralisé destiné aux employés de l'agence. Après authentification, chaque utilisateur accède à un environnement adapté à son rôle et aux permissions qui lui ont été attribuées.

Le système permet notamment de regrouper la gestion des clients, des dossiers, des tâches, des demandes de visa, de la billetterie, des paiements, des guides et de leurs missions.

Il intègre également des mécanismes permettant de suivre certaines actions effectuées dans le système, de transmettre des notifications aux utilisateurs concernés et de consulter des informations synthétiques relatives à l'activité.

1.6 Principe de fonctionnement général

Le fonctionnement de l'application repose sur le dossier client, qui constitue un élément central dans l'organisation des différentes opérations.

Lorsqu'un client sollicite un service, son dossier peut être créé puis suivi tout au long de son traitement. Selon la nature du service demandé, différentes opérations peuvent ensuite lui être associées.

Les employés concernés interviennent selon leurs responsabilités et leurs droits d'accès. Le système permet ainsi de conserver les informations liées à un même dossier dans un environnement commun tout en contrôlant les actions accessibles à chaque profil.

La gestion financière fait également partie du processus. Selon la situation du client et les règles appliquées, une avance peut être nécessaire avant la poursuite du traitement du dossier. Le système permet de conserver et de suivre les opérations de paiement associées.

1.7 Gestion multi-agences

L'un des aspects importants du projet concerne la prise en charge de plusieurs agences au sein d'une même application.

Chaque employé peut être rattaché à une agence. Pour les profils standards, l'accès aux informations est limité aux données appartenant à leur agence d'affectation. Cette règle permet d'éviter qu'un employé puisse consulter ou modifier directement des informations appartenant à une autre agence sans autorisation.

Certains profils disposant de responsabilités globales peuvent, quant à eux, bénéficier d'une visibilité plus large sur les données du système.

Ce principe permet de conserver une application unique tout en respectant la séparation organisationnelle entre les différentes agences.

1.8 Apports attendus de la solution

La mise en place du système vise principalement à améliorer l'organisation interne et le suivi des opérations.

La centralisation des informations doit permettre de réduire la dispersion des données et de faciliter leur consultation par les personnes autorisées. La gestion des rôles et des permissions vise également à mieux contrôler les responsabilités de chaque utilisateur.

Le système permet par ailleurs de disposer d'un suivi plus structuré des dossiers, des tâches et des différentes opérations associées, tout en offrant une architecture capable d'évoluer avec les besoins futurs de l'entreprise.

1.9 Conclusion

Ce chapitre a permis de présenter le contexte général de **BMT Tourisme**, son organisation fonctionnelle ainsi que les principales motivations ayant conduit à la réalisation du système de gestion interne.

La compréhension de cet environnement constitue une étape essentielle avant la définition détaillée des fonctionnalités de la solution.

Le chapitre suivant sera consacré à **l'étude de l'existant et à l'analyse des besoins**, afin d'identifier plus précisément les limites de l'organisation actuelle, les besoins des différents acteurs et les exigences auxquelles l'application devra répondre.

Chapitre 2

Étude de l'Existant et Analyse des Besoins

2.1 Introduction

Avant de procéder à la conception et au développement d'une solution informatique, il est essentiel d'identifier les besoins auxquels elle doit répondre et de comprendre l'environnement dans lequel elle sera utilisée.

Ce chapitre présente une analyse du fonctionnement métier considéré dans le cadre de BMT Tourisme. Il met en évidence les principales contraintes liées au traitement des informations et définit les besoins des différents utilisateurs du futur système.

Cette analyse constitue la base des spécifications fonctionnelles présentées dans le chapitre suivant.

2.2 Étude de l'existant

L'activité d'une agence de voyage nécessite la manipulation quotidienne d'un volume important d'informations provenant de plusieurs services. Un même client peut être concerné par différentes opérations telles qu'une demande de visa, une réservation de billet, un paiement ou encore une prestation nécessitant l'intervention d'un guide.

Le traitement d'un dossier implique ainsi plusieurs acteurs qui doivent pouvoir consulter les informations nécessaires et assurer le suivi des opérations relevant de leurs responsabilités.

Dans une organisation où ces informations ne sont pas regroupées dans un système centralisé, le suivi global du dossier peut devenir plus complexe. Les différents intervenants doivent alors assurer une coordination permanente afin de connaître l'état d'avancement du traitement et les opérations restant à effectuer.

2.3 Limites identifiées

L'analyse du fonctionnement métier a permis d'identifier plusieurs difficultés auxquelles une solution centralisée doit répondre.

2.3.1 Dispersion des informations

Les informations relatives à un même client peuvent concerner plusieurs services. L'absence d'un espace centralisé complique leur consultation et peut rendre plus difficile l'obtention d'une vision complète du dossier.

2.3.2 Suivi des dossiers

Un dossier peut passer par différentes étapes et nécessiter l'intervention de plusieurs employés. Il devient donc nécessaire de pouvoir identifier facilement son état actuel et les opérations qui lui sont associées.

2.3.3 Coordination entre les services

Les commerciaux, responsables visa, agents de billetterie, comptables et guides interviennent sur des aspects différents du traitement. Une mauvaise circulation de l'information entre ces acteurs peut ralentir certaines opérations.

2.3.4 Gestion des responsabilités

Chaque utilisateur ne doit pas disposer des mêmes droits dans le système. Certaines opérations doivent être accessibles uniquement aux employés autorisés en fonction de leur rôle et de leurs responsabilités.

2.3.5 Gestion de plusieurs agences

La présence de plusieurs agences implique également une séparation des données. Un employé rattaché à une agence ne doit pas automatiquement disposer d'un accès aux informations d'une autre agence.

Une solution adaptée doit donc assurer cette séparation tout en permettant aux profils disposant de responsabilités globales d'obtenir une vision plus large.

2.3.6 Suivi financier

Le traitement d'un dossier peut dépendre du versement d'une avance ou d'un paiement. Il est ainsi nécessaire de conserver une trace des opérations financières et de pouvoir connaître le montant payé ainsi que le montant restant.

2.4 Identification des acteurs

Le système est destiné à plusieurs catégories d'utilisateurs ayant des responsabilités différentes.

Acteur	Responsabilité principale
Administrateur	Administration du système, des utilisateurs, des rôles, des permissions et des paramètres de gestion.
Directeur	Consultation et supervision de l'activité globale ainsi que des différentes agences.
Commercial	Gestion des clients, création des dossiers et suivi des opérations commerciales associées.
Responsable Visa	Gestion et suivi des demandes de visa associées aux dossiers.
Agent Billetterie	Gestion des réservations, des billets et de leur état.
Comptable	Enregistrement et suivi des avances, paiements, soldes et remboursements.
Guide	Consultation et suivi des missions qui lui sont attribuées.
Autre employé	Accès aux fonctionnalités correspondant aux permissions qui lui sont attribuées.

Table 2.1 – Principaux acteurs du système

2.5 Analyse des besoins fonctionnels

Les besoins fonctionnels représentent les opérations que le système doit permettre aux utilisateurs autorisés d'effectuer.

À ce stade, ils sont présentés de manière globale. Leur description détaillée sera abordée dans le chapitre consacré aux spécifications fonctionnelles.

2.5.1 Authentification et gestion des accès

Le système doit permettre aux employés de s'authentifier avant d'accéder à l'application.

Les fonctionnalités proposées doivent ensuite dépendre du rôle et des permissions de l'utilisateur connecté. Les droits peuvent provenir du rôle attribué à l'employé ou de permissions spécifiques qui lui sont accordées.

2.5.2 Gestion des clients

Le système doit permettre aux utilisateurs autorisés de rechercher, consulter, créer et mettre à jour les informations relatives aux clients.

Les informations principales concernent notamment l'identité, les coordonnées et certains éléments permettant de faciliter le traitement des dossiers.

2.5.3 Gestion des dossiers

Un client peut disposer d'un ou plusieurs dossiers correspondant aux services demandés.

Le système doit permettre de créer un dossier, de suivre son statut, de conserver les informations nécessaires à son traitement et de regrouper les différentes opérations qui lui sont associées.

2.5.4 Gestion documentaire

Certains dossiers nécessitent la collecte et le suivi de documents.

L'application doit permettre d'associer des documents à un dossier et de suivre leur état, par exemple lorsqu'un document est reçu, validé, manquant ou refusé.

2.5.5 Gestion des tâches

Le système doit permettre de créer et d'attribuer des tâches aux employés autorisés.

Chaque tâche doit pouvoir être associée à un dossier et disposer d'informations permettant d'en suivre la priorité, le responsable et l'état d'avancement.

2.5.6 Gestion des paiements

L'application doit permettre d'enregistrer les opérations financières associées aux dossiers.

Ces opérations peuvent notamment correspondre à une avance, un paiement, un règlement de solde ou un remboursement.

Le système doit également permettre de déterminer le montant déjà payé et le montant restant à régler.

2.5.7 Gestion des visas

Pour les dossiers concernés, le responsable visa doit pouvoir suivre les différentes étapes d'une demande.

Le système doit notamment permettre de conserver les informations relatives à la destination, au type de visa, aux rendez-vous, au dépôt et au résultat de la demande.

2.5.8 Gestion de la billetterie

Les utilisateurs autorisés doivent pouvoir enregistrer et suivre les billets associés aux dossiers.

Les informations concernent notamment le type de billet, la compagnie, le trajet, les dates, la référence de réservation, le prix et le statut.

2.5.9 Gestion des guides et des missions

Le système doit permettre de gérer les guides et d'associer des missions aux dossiers concernés.

Un guide doit pouvoir consulter les missions qui lui sont attribuées et suivre leur état selon les permissions dont il dispose.

2.5.10 Notifications et traçabilité

L'application doit permettre de transmettre des notifications aux employés concernés afin de faciliter le suivi de certaines opérations.

Une traçabilité des actions importantes doit également permettre de conserver un historique des opérations réalisées dans le système.

2.5.11 Tableau de bord et rapports

Le système doit proposer des informations synthétiques permettant aux utilisateurs autorisés de suivre l'activité.

Les informations présentées doivent respecter les droits d'accès et le périmètre de l'agence de l'utilisateur connecté.

2.6 Règles de gestion principales

L'analyse des besoins permet également d'identifier plusieurs règles de gestion importantes.

- Chaque employé est associé à un compte utilisateur et peut être rattaché à une agence et à un rôle.
- Les fonctionnalités accessibles dépendent des permissions accordées à l'utilisateur.
- Les employés standards ne peuvent accéder qu'aux données appartenant à leur agence d'affectation.
- Les profils disposant d'un accès global peuvent consulter les données des différentes agences.
- Un dossier appartient à un client et à une agence.
- Plusieurs opérations peuvent être associées à un même dossier : documents, tâches, paiements, billets ou missions.
- Selon la situation du client et les règles appliquées, le traitement d'un dossier peut nécessiter le versement préalable d'une avance.
- Certains clients peuvent être autorisés à poursuivre le traitement sans avance préalable.
- Les opérations sensibles doivent être protégées par des contrôles d'autorisation côté serveur.

2.7 Besoins non fonctionnels

Au-delà des fonctionnalités métier, la solution doit satisfaire plusieurs exigences non fonctionnelles.

2.7.1 Sécurité

L'application doit empêcher les utilisateurs non autorisés d'accéder aux fonctionnalités ou aux données qui ne correspondent pas à leurs droits.

Les contrôles doivent être appliqués au niveau de l'application et ne doivent pas reposer uniquement sur l'affichage ou le masquage des éléments de l'interface.

2.7.2 Ergonomie

L'interface doit être claire, cohérente et suffisamment intuitive pour faciliter l'utilisation quotidienne de l'application par les différents profils d'employés.

2.7.3 Performance

Les opérations courantes, telles que la consultation des dossiers, la recherche de clients ou l'affichage du tableau de bord, doivent être exécutées dans des délais adaptés à une utilisation professionnelle normale.

2.7.4 Fiabilité

Le système doit assurer la cohérence des informations enregistrées et limiter les opérations susceptibles de produire des données incorrectes ou incohérentes.

2.7.5 Évolutivité

L'architecture doit permettre l'ajout futur de nouvelles agences, de nouveaux rôles, de nouvelles permissions ou de nouvelles fonctionnalités sans nécessiter une refonte complète du système.

2.7.6 Maintenabilité

Le code doit être organisé de manière modulaire afin de faciliter sa compréhension, sa maintenance et son évolution.

2.8 Synthèse des besoins

L'analyse réalisée met en évidence la nécessité d'une plateforme centralisée capable de prendre en charge plusieurs domaines métier tout en assurant une séparation claire des responsabilités.

La solution doit donc combiner trois dimensions principales : la centralisation des données, la coordination des opérations et le contrôle des accès.

Ces éléments constituent la base fonctionnelle sur laquelle repose la conception du système de gestion interne de BMT Tourisme.

2.9 Conclusion

Ce chapitre a permis d'étudier le contexte de gestion considéré, d'identifier les principales limites auxquelles la solution doit répondre et de définir les besoins fonctionnels et non fonctionnels du système.

Les différents acteurs ainsi que les principales règles de gestion ont également été identifiés.

Le chapitre suivant sera consacré aux **spécifications fonctionnelles**, afin de détailler de manière plus précise le comportement attendu des différents modules de l'application.

Chapitre 3

Spécifications Fonctionnelles

3.1 Introduction

À la suite de l'analyse des besoins, il est nécessaire de traduire les attentes identifiées en fonctionnalités précises. Les spécifications fonctionnelles décrivent le comportement attendu du système indépendamment des choix techniques retenus pour son implémentation.

Ce chapitre présente les principales fonctionnalités du système de gestion interne de BMT Tourisme ainsi que les interactions entre les différents modules.

3.2 Authentification et accès au système

L'accès à l'application est réservé aux utilisateurs autorisés. Chaque employé dispose d'un compte lui permettant de s'authentifier avant d'accéder aux fonctionnalités du système.

Après authentification, les éléments accessibles dépendent du profil de l'utilisateur, de son rôle, des permissions qui lui sont attribuées et, lorsque cela s'applique, de son agence d'affectation.

Le système doit notamment permettre :

— l'authentification sécurisée des utilisateurs ;
— la déconnexion de l'utilisateur ;
— la vérification de l'état actif du compte employé ;
— l'adaptation des fonctionnalités selon les permissions ;
— la limitation de l'accès aux données selon l'agence.

3.3 Gestion des employés, rôles et permissions

Le système repose sur une gestion flexible des droits d'accès. Un employé peut être associé à une agence et à un rôle correspondant à sa fonction au sein de l'organisation.

Un rôle peut regrouper plusieurs permissions métier. Des permissions supplémen-

taires peuvent également être accordées directement à un employé lorsque ses responsabilités nécessitent un accès particulier.

Cette organisation permet d'éviter une dépendance stricte entre un poste et un ensemble figé de fonctionnalités.

Les opérations de gestion comprennent notamment :

— la gestion des employés ;
— l'affectation d'un employé à une agence ;
— l'attribution d'un rôle ;
— la gestion des permissions associées aux rôles ;
— l'attribution de permissions spécifiques ;
— l'activation ou la désactivation d'un employé.

3.4 Gestion des agences

L'application prend en charge une organisation multi-agences.

Chaque agence peut être enregistrée avec ses principales informations et peut regrouper plusieurs employés, clients et dossiers.

Pour les utilisateurs standards, les données accessibles sont limitées à l'agence à laquelle ils sont affectés. Les utilisateurs disposant de droits globaux peuvent, selon leur profil, accéder aux informations des différentes agences.

Cette règle doit être appliquée au niveau du traitement des requêtes afin qu'un utilisateur ne puisse pas contourner les restrictions simplement en modifiant une adresse ou une requête envoyée au serveur.

3.5 Gestion des clients

Le module de gestion des clients constitue le point de départ de nombreuses opérations.

L'utilisateur autorisé peut rechercher un client existant avant d'enregistrer une nouvelle fiche afin de limiter les doublons.

Une fiche client contient notamment les informations d'identification et de contact nécessaires à son suivi.

Le système permet principalement :

— de consulter la liste des clients accessibles ;

— de rechercher un client ;
— d'ajouter un nouveau client ;
— de consulter ses informations ;
— de modifier ses informations ;
— d'identifier son type de relation avec l'agence ;
— de déterminer s'il peut être autorisé à poursuivre certaines opérations sans avance préalable.

Chaque client est également associé à l'agence dans laquelle il est pris en charge.

3.6 Gestion des dossiers

Le dossier représente l'un des éléments centraux du système. Il regroupe les informations nécessaires au traitement d'une demande effectuée par un client.

Chaque dossier possède un numéro unique et est associé à un client, à une agence ainsi qu'à l'employé ayant procédé à sa création.

Le système permet de gérer plusieurs types de services, notamment les dossiers relatifs au visa, à la billetterie, aux voyages organisés, aux réservations d'hôtel ou à d'autres prestations.

Un dossier peut évoluer à travers plusieurs états :

Nouveau → En cours → En attente → Terminé

Selon les circonstances, un dossier peut également être bloqué ou annulé.

Le dossier constitue un point de regroupement pour plusieurs informations : documents, tâches, paiements, billets et missions.

3.7 Gestion des avances et traitement des dossiers

Le système prend en considération les règles liées aux avances versées par les clients.

Selon la situation du client, une avance peut être nécessaire avant de poursuivre le traitement d'un dossier.

Le système doit donc distinguer notamment :

— les dossiers nécessitant une avance ;
— les dossiers ne nécessitant pas d'avance ;

— les clients bénéficiant d'une autorisation de traitement sans avance préalable.

Cette règle permet de déterminer si le dossier remplit les conditions nécessaires pour poursuivre son traitement.

3.8 Gestion des documents

Plusieurs prestations nécessitent la collecte de documents auprès du client.

Le système permet d'associer des documents à un dossier et de suivre leur état durant le traitement.

Un document peut notamment avoir l'un des statuts suivants :

— Reçu ;
— Validé ;
— Manquant ;
— Refusé.

Un commentaire peut également être associé au document afin de conserver des informations complémentaires concernant son traitement.

3.9 Gestion des tâches

Le module des tâches facilite la coordination entre les employés.

Une tâche est associée à un dossier et peut être attribuée à un responsable. Elle contient un titre, une description, une priorité, un statut et éventuellement une date d'échéance.

Les priorités permettent de distinguer les tâches :

Faible – Moyenne – Haute – Urgente

Les statuts permettent quant à eux de suivre leur progression depuis leur création jusqu'à leur achèvement ou leur annulation.

L'attribution des tâches reste soumise aux permissions de l'utilisateur ainsi qu'aux règles de séparation entre agences.

3.10 Gestion des paiements

Le module financier permet de conserver les opérations de paiement associées aux dossiers.

Quatre catégories principales sont prises en charge :

— l'avance ;
— le paiement ;
— le règlement du solde ;
— le remboursement.

Pour chaque opération, le système conserve notamment le montant, le mode de paiement, une éventuelle référence, un commentaire et l'employé ayant enregistré l'opération.

À partir de ces informations, le système peut déterminer le montant net payé ainsi que le reste à payer sur le dossier.

3.11 Gestion des visas

Pour les dossiers nécessitant une procédure de visa, un suivi spécifique est disponible.

La demande peut évoluer à travers différentes étapes telles que :

Documents manquants → Documents complets → Dossier prêt → Déposé → En traitement

Elle peut ensuite aboutir à une décision d'acceptation ou de refus.

Le module conserve également les informations nécessaires au suivi, notamment le pays de destination, le type de visa, la date de rendez-vous, la date de dépôt, le numéro de demande et la date du résultat lorsqu'ils sont disponibles.

3.12 Gestion de la billetterie

Le module de billetterie permet d'associer une ou plusieurs réservations à un dossier.

Le système prend en charge différents types de billets, notamment l'avion, le train et le bus.

Chaque billet peut contenir :

— la compagnie ;

— la référence de réservation ;
— la ville de départ ;
— la ville d'arrivée ;
— la date de départ ;
— une éventuelle date de retour ;
— le prix ;
— le statut de la réservation.

Le statut permet de distinguer une réservation en attente, réservée, émise, annulée ou modifiée.

3.13 Gestion des guides et des missions

Les employés exerçant la fonction de guide peuvent disposer d'informations complémentaires telles que les langues maîtrisées, leur spécialité et leur disponibilité.

Une mission peut être associée à un guide et à un dossier.

Elle précise notamment le lieu, les dates de début et de fin, le statut ainsi qu'un éventuel commentaire.

Le guide peut consulter les missions qui lui sont attribuées et effectuer les actions autorisées par ses permissions.

3.14 Notifications

Le système intègre un mécanisme de notification destiné à faciliter la transmission d'informations aux utilisateurs.

Une notification est associée à un employé et contient un titre, un message ainsi qu'un état permettant de déterminer si elle a été consultée.

Ce mécanisme contribue à améliorer le suivi des événements nécessitant l'attention d'un utilisateur.

3.15 Historique des actions

Afin d'améliorer la traçabilité, certaines opérations importantes peuvent être enregistrées dans un historique.

Une entrée d'historique permet notamment de conserver :

— l'employé concerné ;
— l'action réalisée ;
— le module concerné ;
— l'objet éventuellement associé ;
— une description ;
— la date de l'action.

Ce mécanisme facilite la consultation des actions réalisées dans l'application.

3.16 Tableau de bord

Après authentification, l'utilisateur accède à un tableau de bord présentant les informations correspondant à son profil.

Le tableau de bord constitue un point d'entrée vers les différents modules et peut présenter des indicateurs synthétiques concernant l'activité accessible à l'utilisateur.

Le contenu affiché respecte les permissions ainsi que le périmètre de l'agence de l'employé.

3.17 Rapports et indicateurs

Les utilisateurs autorisés peuvent consulter des indicateurs calculés à partir des données enregistrées dans le système.

Ces indicateurs peuvent concerner notamment :

— les clients ;
— les dossiers ;
— les opérations financières ;
— les visas ;
— la billetterie ;
— les missions des guides ;
— l'activité des différentes agences pour les profils disposant d'une visibilité globale.

L'objectif est de fournir une vue synthétique facilitant le suivi de l'activité et la consultation des principales informations de gestion.

3.18 Synthèse du fonctionnement

Le fonctionnement global du système peut être résumé par un processus dans lequel un client est identifié ou créé, puis associé à un dossier. Ce dossier devient ensuite le support des différentes opérations nécessaires à la prestation demandée.

Les employés interviennent sur ces opérations selon leur rôle, leurs permissions et leur agence d'affectation. Les données générées au cours du traitement permettent ensuite d'assurer le suivi opérationnel et d'alimenter les indicateurs de l'application.

3.19 Conclusion

Ce chapitre a détaillé les principales fonctionnalités attendues du système de gestion interne de BMT Tourisme ainsi que les règles qui structurent les interactions entre ses différents modules.

Ces spécifications constituent une base pour définir la manière dont la solution doit être organisée sur le plan technique.

Le chapitre suivant sera consacré à **l'architecture du système et aux choix technologiques**, en présentant notamment l'organisation de l'application et les technologies retenues pour sa réalisation.

Chapitre 4

Architecture du Système et Choix Techniques

4.1 Introduction

Après avoir défini les besoins et les fonctionnalités attendues, il convient de présenter l'organisation technique retenue pour la réalisation du système.

Ce chapitre décrit l'architecture générale de l'application, les principales technologies utilisées ainsi que l'organisation du projet. L'objectif est de justifier les choix techniques adoptés pour obtenir une solution modulaire, maintenable et évolutive.

4.2 Architecture générale de l'application

Le système de gestion interne de BMT Tourisme a été développé sous la forme d'une application web centralisée.

Cette approche permet aux différents utilisateurs d'accéder au système à travers une interface commune, tout en adaptant les fonctionnalités disponibles selon leurs rôles, leurs permissions et leur agence d'affectation.

L'application repose sur trois éléments principaux :

— une couche de présentation destinée aux utilisateurs ;
— une couche applicative assurant le traitement de la logique métier ;
— une couche de données assurant la persistance des informations.

Cette séparation permet d'organiser le système de manière structurée et facilite son évolution.

4.3 Framework Django

Le développement de l'application repose principalement sur **Django**, un framework web basé sur le langage Python.

Django fournit plusieurs composants utiles au développement d'applications web structurées, notamment la gestion des modèles, des routes, des vues, des formulaires, des templates, de l'authentification et des interactions avec la base de données.

Son utilisation permet également d'organiser l'application en plusieurs modules indépendants correspondant aux différents domaines fonctionnels du projet.

4.4 Architecture MVT

Django repose sur une organisation couramment appelée **MVT – Model, View, Template**.

4.4.1 Model

La partie **Model** représente les données manipulées par l'application ainsi que leurs relations.

Les modèles permettent notamment de représenter les agences, les employés, les clients, les dossiers, les paiements, les visas, les billets, les tâches et les missions.

Django assure ensuite la correspondance entre ces modèles Python et les tables de la base de données.

4.4.2 View

La partie **View** contient la logique permettant de traiter les requêtes envoyées par les utilisateurs.

Les vues récupèrent les données nécessaires, appliquent les contrôles d'accès et les règles métier, puis transmettent les informations appropriées à l'interface.

4.4.3 Template

Les **Templates** représentent la couche d'affichage.

Ils permettent de générer les pages HTML présentées à l'utilisateur et d'adapter certains éléments de l'interface en fonction des données, du rôle et des permissions disponibles.

Le fonctionnement général peut ainsi être résumé de la manière suivante :

Utilisateur $\longrightarrow$ **URL** $\longrightarrow$ **View** $\longrightarrow$ **Model / Base de données** $\longrightarrow$ **Template** $\longrightarrow$ **Utilisateur**

4.5 Organisation modulaire du projet

Afin d'éviter une application monolithique difficile à maintenir, le projet Django a été divisé en plusieurs applications spécialisées.

Application	Responsabilité
accounts	Gestion des employés, rôles, permissions, authentification et contrôle des accès.
agencies	Gestion des différentes agences de BMT Tourisme.
clients	Gestion des informations relatives aux clients.
dossiers	Gestion des dossiers clients et de leurs documents.
tasks	Gestion et affectation des tâches.
payments	Gestion des avances, paiements, soldes et remboursements.
visa	Suivi des demandes et procédures de visa.
ticketing	Gestion des réservations et des billets.
guides	Gestion des guides et de leurs missions.
notifications	Gestion des notifications et de l'historique des actions.
reports	Calcul et présentation des indicateurs de suivi.

Table 4.1 – Organisation des applications Django du projet

Cette séparation permet de regrouper les fonctionnalités appartenant au même domaine métier et facilite la maintenance du code.

4.6 Langage Python

Le langage **Python** est utilisé pour le développement de la partie serveur de l'application.

Il permet notamment d'implémenter :

— les règles métier ;
— les modèles de données ;
— les traitements des formulaires ;
— les contrôles d'accès ;
— les calculs liés aux paiements ;
— les traitements nécessaires aux rapports et indicateurs.

Son intégration avec Django permet de structurer les différentes fonctionnalités dans une architecture cohérente.

4.7 Base de données MySQL

La persistance des données repose sur le système de gestion de base de données relationnelle **MySQL**.

La base de données utilisée par l'application permet de stocker de manière structurée les différentes informations métier.

Django communique avec MySQL à travers son système **ORM – Object Relational Mapping**. Celui-ci permet de manipuler les données à partir des modèles Python sans devoir écrire directement les requêtes SQL pour la majorité des opérations courantes.

L'utilisation d'une base relationnelle est adaptée au projet en raison des nombreuses relations existant entre les entités, par exemple entre un client et ses dossiers ou entre un dossier et ses paiements.

4.8 Technologies Front-End

L'interface utilisateur a été réalisée à partir de plusieurs technologies web complémentaires.

4.8.1 HTML

HTML permet de définir la structure des différentes pages de l'application : formulaires, tableaux, menus, cartes et zones de contenu.

4.8.2 CSS

CSS est utilisé pour personnaliser l'apparence de l'application, améliorer la lisibilité des interfaces et assurer une identité visuelle cohérente.

4.8.3 Bootstrap

Le framework **Bootstrap** facilite la création d'interfaces responsives et fournit plusieurs composants réutilisables tels que les formulaires, tableaux, boutons et éléments de navigation.

4.8.4 JavaScript

JavaScript complète l'interface en permettant d'ajouter des comportements interactifs lorsque cela est nécessaire.

4.9 Principe du tableau de bord partagé

L'application utilise une structure d'interface commune aux différents profils plutôt que de créer une application indépendante pour chaque type d'employé.

Après authentification, l'utilisateur accède à une interface générale dont le contenu et les éléments de navigation sont adaptés selon ses permissions.

Cette approche permet de conserver une expérience utilisateur cohérente tout en respectant les responsabilités de chaque profil.

Il est toutefois important de préciser que le masquage d'un élément dans l'interface ne constitue pas à lui seul un mécanisme de sécurité. Les autorisations sont également vérifiées au niveau du serveur avant l'exécution des opérations protégées.

4.10 Gestion des rôles et permissions

Le contrôle des accès repose sur une combinaison de rôles et de permissions métier.

Un rôle regroupe un ensemble de permissions correspondant généralement aux responsabilités d'un profil professionnel.

Le système permet également d'attribuer directement des permissions spécifiques à un employé. Cette approche offre davantage de flexibilité lorsqu'un utilisateur doit disposer d'un droit particulier sans modifier les permissions de l'ensemble de son rôle.

Avant l'accès à une fonctionnalité protégée, le système vérifie donc les droits de l'utilisateur connecté.

4.11 Gestion du périmètre des agences

La sécurité des données ne repose pas uniquement sur les permissions fonctionnelles.

Le système applique également une règle de portée permettant de déterminer quelles données un utilisateur peut consulter.

Pour les profils standards, les requêtes sont limitées aux informations appartenant à leur agence d'affectation.

Les profils disposant d'un accès global peuvent consulter un périmètre plus large selon leurs responsabilités.

Cette vérification est réalisée côté serveur afin qu'une modification manuelle d'une URL ou d'une requête ne permette pas de contourner la restriction.

4.12 Environnement de développement

Le projet a été réalisé dans un environnement de développement local.

Les principaux éléments utilisés sont :

Élément	Technologie
Langage Back-End	Python
Framework Web	Django
Base de données	MySQL
Serveur MySQL local	WAMP
Front-End	HTML, CSS, Bootstrap, JavaScript
Architecture	MVT

Table 4.2 – Principales technologies utilisées

4.13 Justification des choix techniques

Les technologies retenues répondent aux besoins principaux du projet.

Django facilite la structuration d'une application composée de plusieurs domaines métier et fournit des mécanismes intégrés pour l'authentification, la gestion des données et la sécurité des requêtes.

MySQL permet de représenter efficacement les nombreuses relations entre les données métier, tandis que Bootstrap et les technologies web utilisées côté interface facilitent la réalisation d'une application accessible depuis un navigateur et adaptable à différentes tailles d'écran.

L'organisation modulaire du projet permet également d'isoler les responsabilités et de préparer l'application à de futures évolutions.

4.14 Conclusion

Ce chapitre a présenté l'architecture générale de la solution, l'organisation MVT adoptée par Django ainsi que les principales technologies utilisées pour le développement

de l'application.

L'organisation modulaire du projet et les mécanismes de contrôle des accès constituent des éléments importants de l'architecture retenue.

Après avoir présenté la structure technique de la solution, le chapitre suivant sera consacré à **la conception UML du système**, afin de représenter de manière formelle les acteurs, les entités et les principaux comportements de l'application.

Chapitre 5

Conception UML du Système

5.1 Introduction

Après l'analyse des besoins et la définition de l'architecture générale de la solution, une phase de conception est nécessaire afin de représenter de manière structurée le fonctionnement du système avant et pendant sa réalisation.

Pour cette étape, le langage **UML (Unified Modeling Language)** a été utilisé. Il permet de représenter le système selon plusieurs points de vue complémentaires, notamment les interactions avec les utilisateurs, la structure des données et le comportement des différents processus métier.

Dans le cadre de ce projet, cinq diagrammes principaux ont été retenus : le diagramme de cas d'utilisation, le diagramme de classes, le diagramme de séquence, le diagramme d'activité et le diagramme d'états.

5.2 Diagramme de cas d'utilisation

Le diagramme de cas d'utilisation permet de représenter les principales fonctionnalités offertes par le système ainsi que les acteurs qui interagissent avec celles-ci.

Dans le système de gestion interne de BMT Tourisme, plusieurs profils peuvent intervenir, notamment l'administrateur, le directeur, le commercial, le responsable visa, l'agent de billetterie, le comptable et le guide.

Chaque acteur accède aux fonctionnalités correspondant à ses responsabilités. Cette représentation permet ainsi d'obtenir une vue fonctionnelle globale du système sans entrer dans les détails techniques de son implémentation.

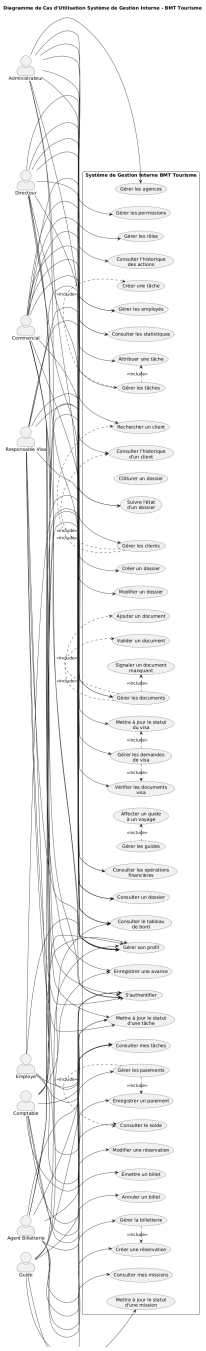


Figure 5.1 – Diagramme de cas d'utilisation du système

Le diagramme met notamment en évidence la séparation des responsabilités entre les différents utilisateurs. L'accès aux fonctionnalités dépend du rôle et des autorisations accordées à chaque employé.

Il permet également de visualiser le caractère centralisé de l'application : plusieurs profils utilisent le même système, mais leurs interactions avec celui-ci diffèrent selon leurs fonctions.

5.3 Diagramme de classes

Le diagramme de classes représente la structure statique du système. Il permet d'identifier les principales entités métier, leurs attributs, leurs opérations ainsi que les relations qui existent entre elles.

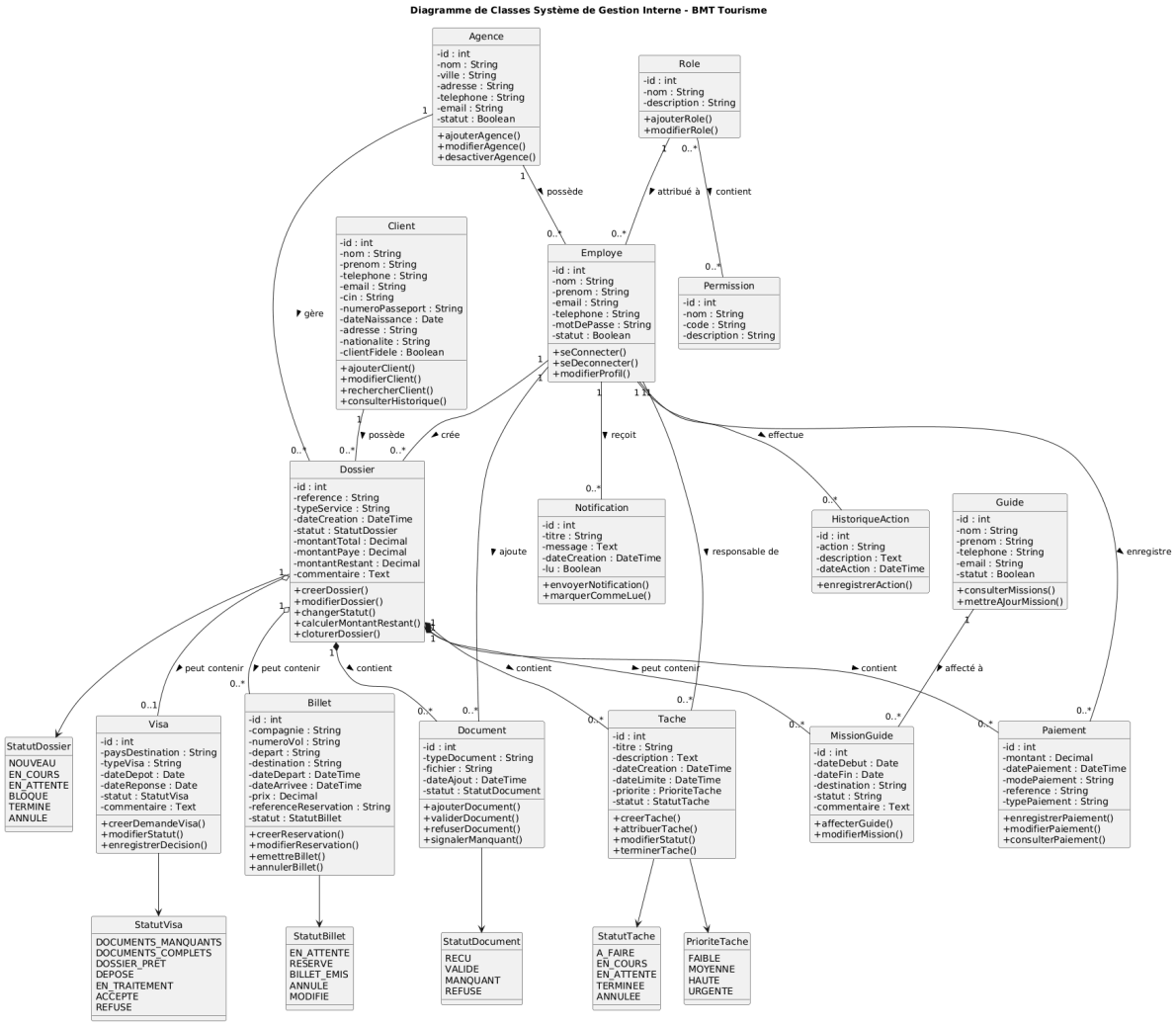


Figure 5.2 – Diagramme de classes du système

La conception repose sur plusieurs classes principales telles que **Agence**, **Employe**, **Role**, **Permission**, **Client**, **Dossier**, **Document**, **Tache**, **Paiement**, **Visa**, **Billet**, **Guide**, **MissionGuide**, **Notification** et **HistoriqueAction**.

Le **Dossier** occupe une position centrale dans cette organisation. Il est rattaché à un client et à une agence et peut être associé à plusieurs éléments nécessaires au traitement de la demande du client.

La séparation entre **Role** et **Permission** permet également de disposer d'une gestion flexible des autorisations. Un rôle peut regrouper plusieurs permissions, tandis que le

système peut accorder des droits spécifiques selon les besoins.

5.4 Diagramme de séquence

Le diagramme de séquence décrit les échanges entre les différents participants d'un scénario en respectant leur ordre chronologique.

Il permet de mieux comprendre comment une action initiée par un utilisateur traverse les différentes composantes du système jusqu'à son traitement et à l'affichage du résultat.

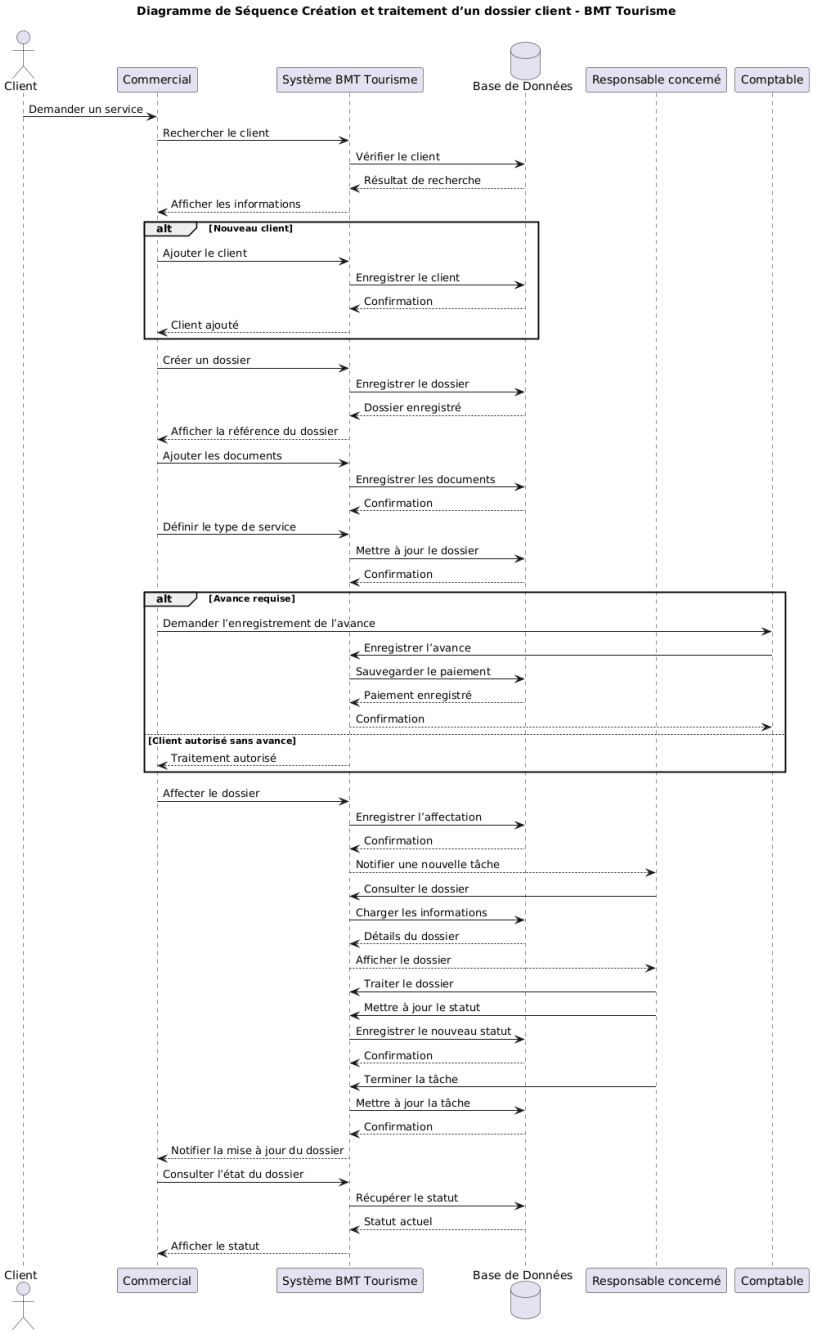


Figure 5.3 – Diagramme de séquence du système

Dans l'application développée, une opération commence généralement par une action de l'utilisateur depuis l'interface. Le système vérifie ensuite les conditions nécessaires, notamment l'authentification et les autorisations, avant d'accéder aux données concernées.

Le résultat du traitement est finalement renvoyé vers l'interface afin d'informer l'utilisateur de l'état de l'opération.

Cette représentation met ainsi en évidence l'enchaînement logique des traitements et la séparation entre l'interaction utilisateur, la logique applicative et l'accès aux données.

5.5 Diagramme d'activité

Le diagramme d'activité permet de représenter l'enchaînement des actions et des décisions intervenant dans un processus métier.

Il est particulièrement adapté à la représentation des traitements comportant plusieurs étapes, conditions ou branches d'exécution.

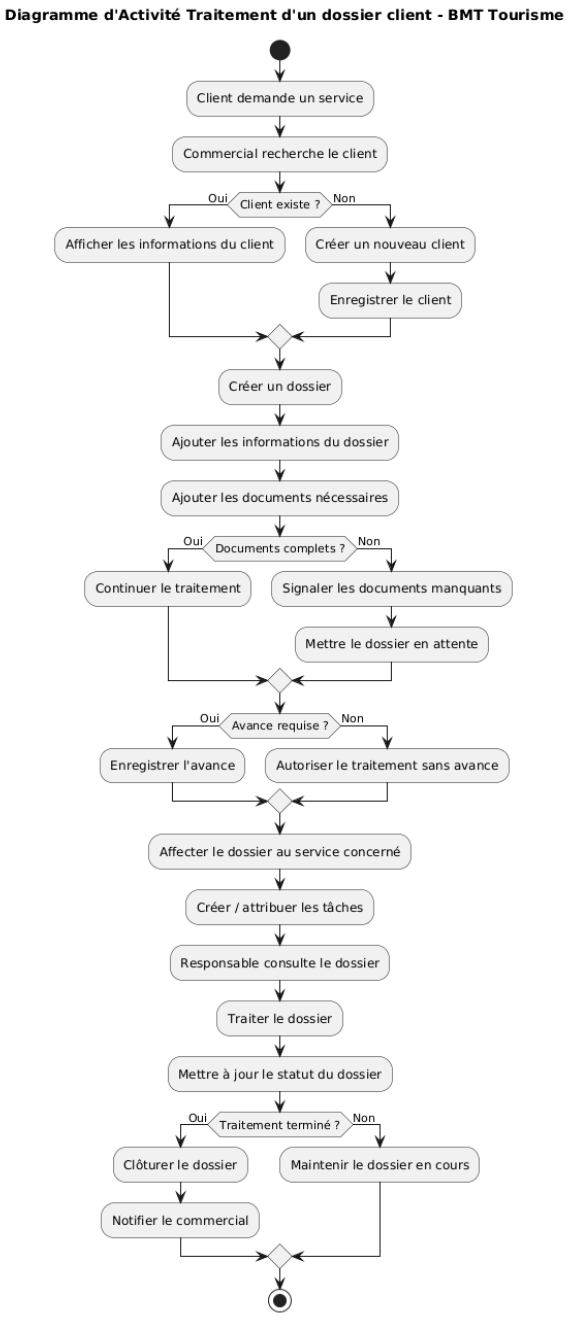


Figure 5.4 – Diagramme d'activité du système

Le diagramme illustre le déroulement d'un processus depuis son déclenchement jusqu'à sa finalisation. Les décisions permettent de représenter les différentes orientations possibles selon les informations disponibles ou les règles métier appliquées.

Cette modélisation permet de vérifier la cohérence du processus avant son implémentation et d'identifier clairement les étapes nécessaires au traitement.

5.6 Diagramme d'états

Le diagramme d'états représente l'évolution d'un objet au cours de son cycle de vie.

Dans une application de gestion, plusieurs objets peuvent changer d'état selon les opérations réalisées. C'est notamment le cas des dossiers, des tâches, des demandes de visa, des billets ou des missions.

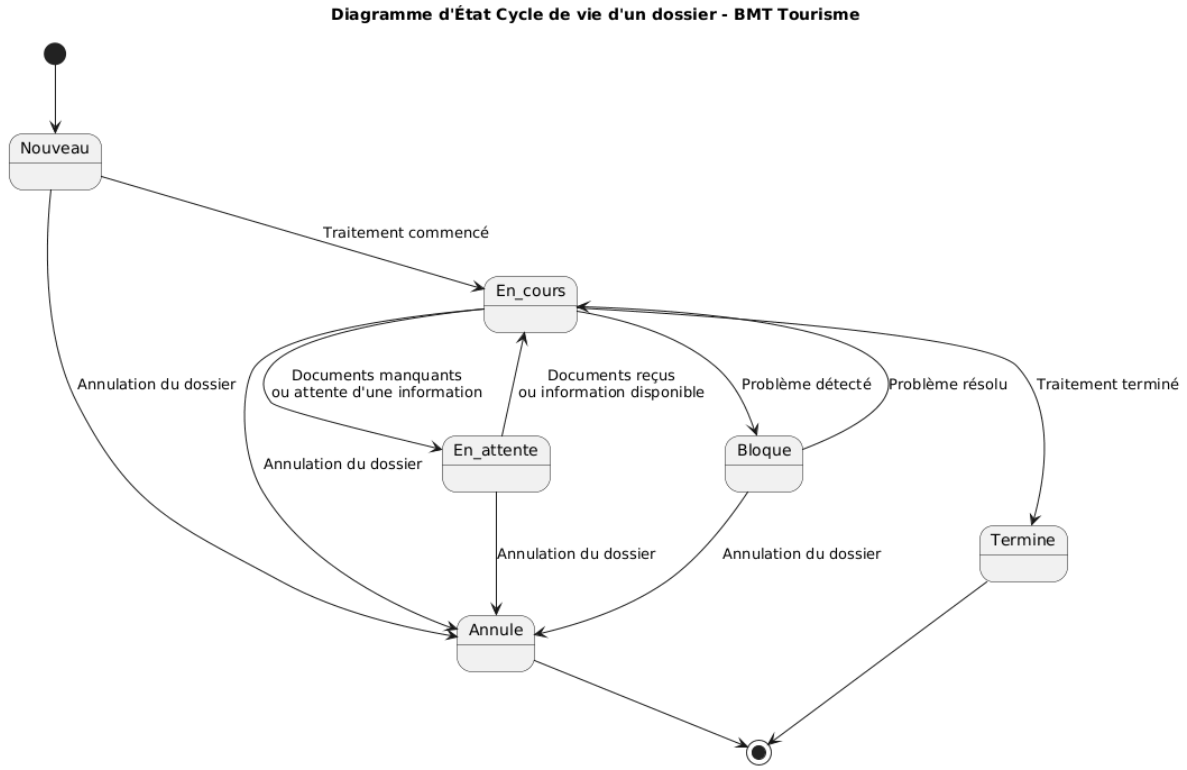


Figure 5.5 – Diagramme d'états du système

La représentation par états permet de définir les différentes situations dans lesquelles peut se trouver l'objet étudié ainsi que les transitions provoquées par les actions réalisées dans l'application.

Elle contribue ainsi à éviter les changements incohérents et à mieux formaliser le cycle de vie des éléments métier.

5.7 Complémentarité des diagrammes

Les cinq diagrammes utilisés ne représentent pas les mêmes aspects du système, mais fournissent des vues complémentaires.

Le diagramme de cas d'utilisation présente principalement **ce que le système permet de faire** et les utilisateurs concernés.

Le diagramme de classes décrit **la structure statique du système** et les relations entre les principales entités.

Le diagramme de séquence représente **l'ordre des interactions** nécessaires à la réalisation d'un scénario.

Le diagramme d'activité décrit **le déroulement d'un processus** et les différentes décisions qui peuvent intervenir.

Enfin, le diagramme d'états représente **l'évolution d'un objet** entre plusieurs états au cours de son cycle de vie.

L'utilisation conjointe de ces représentations permet donc d'obtenir une vision plus complète de la solution avant d'aborder la structure détaillée des données.

5.8 Conclusion

Ce chapitre a présenté la conception UML du système de gestion interne de BMT Tourisme à travers cinq représentations complémentaires : le diagramme de cas d'utilisation, le diagramme de classes, le diagramme de séquence, le diagramme d'activité et le diagramme d'états.

Ces diagrammes permettent de formaliser les interactions, la structure et les principaux comportements du système tout en constituant un lien entre l'analyse fonctionnelle et la réalisation technique.

Le chapitre suivant sera consacré à **la conception et à la gestion de la base de données**, afin de présenter la traduction de cette conception en modèles de données, relations et contraintes nécessaires au fonctionnement de l'application.

Chapitre 6

Conception et Gestion de la Base de Données

6.1 Introduction

La base de données constitue un élément central du système de gestion interne de BMT Tourisme. Elle assure la conservation, l'organisation et la cohérence des informations manipulées par les différents modules de l'application.

La conception de cette base doit prendre en considération les relations entre les agences, les employés, les clients, les dossiers et l'ensemble des opérations associées à leur traitement.

Ce chapitre présente la modélisation des données à travers le diagramme de classes, les principales entités du système, leurs relations, les contraintes d'intégrité ainsi que les mécanismes utilisés par Django pour assurer la communication avec la base de données MySQL.

6.2 Choix du système de gestion de base de données

Le système de gestion de base de données retenu pour ce projet est **MySQL**.

MySQL est un système de gestion de base de données relationnelle permettant de structurer les informations sous forme de tables reliées entre elles. Cette approche correspond aux besoins de l'application, dans laquelle de nombreuses relations existent entre les différentes entités métier.

Dans l'environnement de développement du projet, le serveur MySQL est exécuté localement à travers **WAMP**.

La base de données utilisée par l'application est nommée :

bmt_tourisme_db

Elle centralise les données nécessaires au fonctionnement des différents modules du système.

6.3 Interaction entre Django et MySQL

L'application communique avec la base de données à travers **Django ORM (Object Relational Mapping)**.

Ce mécanisme permet de représenter les structures de la base de données sous forme de classes Python. Les modèles définis dans les différentes applications Django sont ensuite traduits en tables et relations dans MySQL.

Le principe général peut être représenté comme suit :

Modèles Django $\longleftrightarrow$ **Django ORM** $\longleftrightarrow$ **Base MySQL**

Cette approche permet d'effectuer les principales opérations sur les données directement depuis Python, tout en bénéficiant des mécanismes proposés par Django pour gérer les relations entre les modèles.

6.4 Modélisation des données

La modélisation des données constitue une étape importante dans la conception du système. Elle permet d'identifier les différentes entités métier, leurs caractéristiques ainsi que les relations qui existent entre elles.

Dans le cadre du système développé pour BMT Tourisme, cette modélisation prend notamment en considération :

— les agences ;
— les employés ;
— les rôles et les permissions ;
— les clients ;
— les dossiers ;
— les documents ;
— les tâches ;
— les paiements ;
— les demandes de visa ;
— les billets ;
— les guides et leurs missions ;
— les notifications ;
— l'historique des actions.

Ces différentes entités permettent de représenter les principales informations nécessaires au fonctionnement de l'application.

6.5 Diagramme de classes

Le diagramme de classes fournit une représentation structurée des principales entités du système, de leurs attributs et des relations qui existent entre elles.

Il constitue un lien important entre l'analyse fonctionnelle de l'application et l'implémentation des modèles de données dans Django.

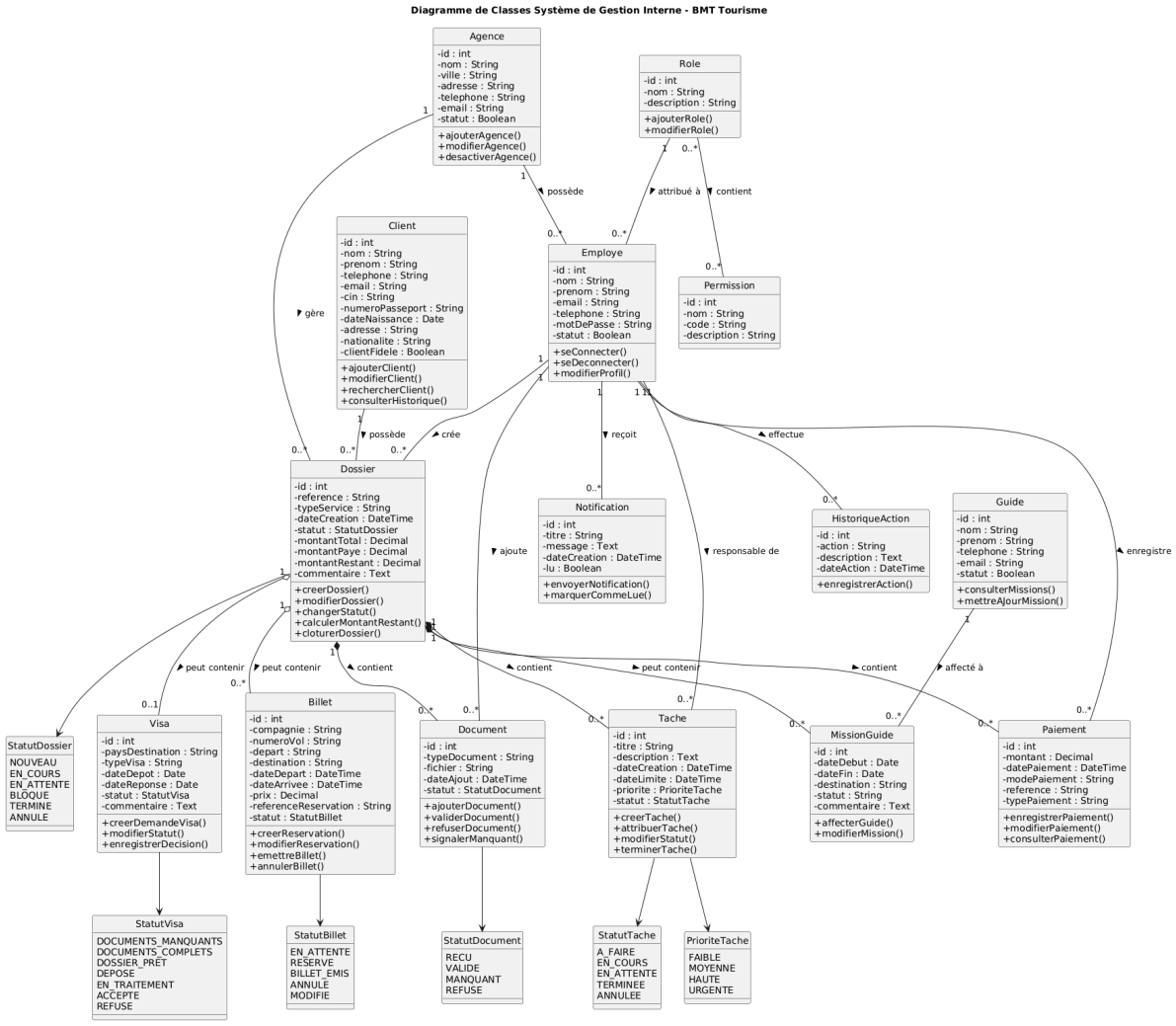


Figure 6.1 – Diagramme de classes du système de gestion interne

Le diagramme met en évidence les principales entités du système, notamment **Agence**, **Employe**, **Role**, **PermissionMetier**, **Client**, **Dossier**, **Document**, **Tache**, **Paiement**, **Visa**, **Billet**, **Guide**, **MissionGuide**, **Notification** et **HistoriqueAction**.

Le modèle **Dossier** occupe une position centrale dans l'organisation des données. Il

permet de regrouper les différentes opérations liées au traitement d'une demande client.

La gestion des accès est représentée principalement par les relations entre les employés, les rôles et les permissions. Cette organisation permet de définir les autorisations de manière flexible.

Enfin, l'association des différentes données aux agences permet d'intégrer la dimension multi-agences directement dans la structure du système.

6.6 Description des principales entités

Les modèles utilisés dans l'application correspondent aux principaux objets métier manipulés par les utilisateurs.

Entité	Description
Agence	Représente une agence de l'organisation et permet notamment de séparer les données selon leur origine.
Employe	Représente le profil professionnel associé à un utilisateur de l'application.
Role	Définit la fonction générale occupée par un employé et regroupe un ensemble de permissions.
PermissionMetier	Représente une autorisation permettant d'accéder à une fonctionnalité spécifique.
Client	Contient les informations d'identification et de contact relatives à un client.
Dossier	Représente une demande ou une prestation associée à un client et constitue l'élément central du traitement métier.
Document	Représente un document associé à un dossier et permet d'en suivre le statut.
Tache	Représente une opération à effectuer dans le cadre du traitement d'un dossier.
Paiement	Enregistre les opérations financières associées à un dossier.
Visa	Contient les informations nécessaires au suivi d'une demande de visa.
Billet	Représente une réservation ou un billet associé à un dossier.
Guide	Contient les informations spécifiques à un employé exerçant la fonction de guide.
MissionGuide	Représente une mission attribuée à un guide et associée à un dossier.
Notification	Permet de transmettre une information à un employé.
HistoriqueAction	Conserve une trace de certaines opérations réalisées dans le système.

Table 6.1 – Description des principales entités du système

6.7 Relations entre les entités

Les différentes entités du système ne fonctionnent pas de manière isolée. Elles sont reliées entre elles afin de représenter les relations existant dans le fonctionnement réel de l'agence.

6.7.1 Relation entre utilisateur et employé

L'authentification repose sur le modèle utilisateur fourni par Django.

Chaque profil **Employe** est associé à un compte utilisateur par une relation de type **OneToOne**.

Cette séparation permet de conserver les informations d'authentification dans le système standard de Django tout en ajoutant les informations professionnelles nécessaires, telles que l'agence, le rôle, le poste ou le numéro de téléphone.

6.7.2 Relation entre agence et employé

Une agence peut regrouper plusieurs employés.

Chaque employé peut être rattaché à une agence. Cette relation joue un rôle important dans le contrôle du périmètre des données accessibles par l'utilisateur.

6.7.3 Relation entre rôle et permissions

Un rôle peut disposer de plusieurs permissions et une même permission peut être utilisée par plusieurs rôles.

Cette relation est donc représentée par une association de type **ManyToMany**.

Le système permet également d'attribuer directement certaines permissions à un employé, indépendamment de celles définies dans son rôle.

6.7.4 Relation entre client et agence

Chaque client peut être associé à l'agence dans laquelle il est pris en charge.

Cette information contribue à l'application des règles de séparation des données entre les différentes agences.

6.7.5 Relation entre client et dossier

Un client peut posséder plusieurs dossiers.

En revanche, chaque dossier est associé à un seul client.

Cette organisation permet de conserver plusieurs prestations ou demandes pour un même client sans dupliquer ses informations générales.

6.7.6 Relation entre dossier et agence

Chaque dossier est associé à une agence.

Cette relation permet de déterminer le périmètre organisationnel auquel appartient le dossier et constitue un élément important du mécanisme de contrôle d'accès multi-agences.

6.7.7 Relations autour du dossier

Le dossier constitue le point de liaison entre plusieurs modules de l'application.

Un dossier peut notamment être associé à plusieurs :

— documents ;
— tâches ;
— paiements ;
— billets ;
— missions de guides.

Une demande de visa peut également être associée au dossier concerné afin de conserver les informations spécifiques à cette procédure.

6.8 Le dossier comme entité centrale

Le modèle **Dossier** occupe une position centrale dans la structure de la base de données.

Chaque dossier contient plusieurs informations nécessaires à son identification et à son traitement, notamment :

— un numéro de dossier unique ;
— le client concerné ;

— l'agence responsable ;
— l'employé ayant créé le dossier ;
— le type de service demandé ;
— le statut du dossier ;
— une description éventuelle ;
— le montant total ;
— l'indication concernant l'obligation d'une avance ;
— la date de création ;
— la date de modification.

Les autres modules viennent ensuite compléter le dossier en lui associant les informations nécessaires à son traitement.

Cette organisation permet d'obtenir une vision centralisée des opérations concernant une demande client.

6.9 Gestion des documents

Les documents sont enregistrés en relation avec un dossier.

Chaque document contient notamment son nom, son statut, un éventuel fichier ainsi qu'un commentaire.

Le statut permet de distinguer plusieurs situations :

— document reçu ;
— document validé ;
— document manquant ;
— document refusé.

L'employé ayant ajouté le document peut également être enregistré, ce qui améliore la traçabilité du traitement.

6.10 Gestion des tâches

Chaque tâche est associée à un dossier.

Elle peut également être liée à un employé responsable ainsi qu'à l'employé ayant procédé à sa création.

Les principales informations conservées sont :

— le titre ;
— la description ;
— le responsable ;
— la priorité ;
— le statut ;
— la date d'échéance ;
— les dates de création et de modification.

Cette structure permet de suivre l'organisation du travail et l'évolution des différentes opérations associées aux dossiers.

6.11 Gestion des données financières

Les opérations financières sont enregistrées séparément sous forme de paiements associés aux dossiers.

Le système distingue plusieurs types d'opérations :

— **Avance** ;
— **Paiement** ;
— **Solde** ;
— **Remboursement**.

Chaque opération peut conserver le montant, le mode de paiement, une référence éventuelle, un commentaire ainsi que l'employé ayant enregistré l'opération.

À partir de ces informations, l'application calcule dynamiquement plusieurs valeurs utiles.

6.11.1 Montant payé

Le montant payé correspond aux opérations financières positives auxquelles sont soustraits les remboursements.

Cette valeur est calculée à partir des paiements existants et n'a donc pas besoin d'être dupliquée dans la base de données.

6.11.2 Reste à payer

Le reste à payer est déterminé à partir du montant total du dossier et du montant net déjà payé.

Le principe de calcul peut être exprimé de manière simplifiée comme suit :

Reste à payer = Montant total - Montant net payé

6.11.3 Gestion des avances

Les avances sont également calculées à partir des opérations financières enregistrées.

Cette information intervient notamment dans la règle permettant de déterminer si le traitement d'un dossier peut se poursuivre lorsqu'une avance est obligatoire.

6.12 Gestion des visas

Une demande de visa est associée au dossier correspondant.

Les informations enregistrées permettent notamment de conserver :

— le responsable ;
— le pays de destination ;
— le type de visa ;
— le statut ;
— la date de dépôt ;
— la date du rendez-vous ;
— la date du résultat ;
— le numéro de demande ;
— un commentaire éventuel.

Cette structure permet de suivre l'évolution de la demande tout au long de son traitement.

6.13 Gestion de la billetterie

Le modèle **Billet** permet d'enregistrer les informations relatives aux réservations associées aux dossiers.

Chaque billet peut notamment contenir :

— le type de billet ;
— la compagnie ;
— la référence de réservation ;
— la ville de départ ;
— la ville d'arrivée ;
— la date de départ ;
— la date de retour ;
— le prix ;
— le statut ;
— le responsable du traitement.

Un dossier peut être associé à plusieurs billets, ce qui permet de prendre en charge des situations nécessitant plusieurs réservations.

6.14 Gestion des guides et des missions

Le modèle **Guide** complète les informations d'un employé lorsque celui-ci exerce la fonction de guide.

Il permet notamment de conserver des informations concernant les langues, la spécialité et la disponibilité.

Le modèle **MissionGuide** permet ensuite d'associer un guide à un dossier.

Une mission contient notamment :

— le guide concerné ;
— le dossier ;
— le lieu ;
— la date de début ;
— la date de fin ;
— le statut ;
— un commentaire éventuel.

6.15 Notifications et historique

Deux modèles complémentaires permettent d'améliorer le suivi des événements internes de l'application.

6.15.1 Notification

Une notification est associée à un employé.

Elle contient un titre, un message, une date de création et une information indiquant si elle a été lue.

Ce modèle permet d'informer les utilisateurs des événements nécessitant leur attention.

6.15.2 HistoriqueAction

Le modèle **HistoriqueAction** permet de conserver une trace de certaines opérations importantes.

Une entrée peut notamment contenir :

— l'employé ayant effectué l'action ;
— le type d'action ;
— le module concerné ;
— l'identifiant de l'objet concerné ;
— une description ;
— la date de l'action.

Ce mécanisme améliore la traçabilité des opérations réalisées dans l'application.

6.16 Contraintes et intégrité des données

La cohérence de la base de données repose sur plusieurs contraintes appliquées au niveau des modèles.

6.16.1 Contraintes d'unicité

Certaines informations doivent être uniques afin d'éviter les doublons ou les ambiguïtés.

Le numéro de dossier est notamment défini comme une valeur unique.

Certaines informations d'identification du client, telles que la CIN ou le numéro de passeport lorsqu'ils sont renseignés, peuvent également être soumises à une contrainte d'unicité.

6.16.2 Relations obligatoires

Certaines relations sont indispensables au fonctionnement métier.

Par exemple, un dossier doit être associé à un client et à une agence.

Ces contraintes permettent de limiter la création de données incomplètes ou incohérentes.

6.16.3 Gestion des suppressions

Django permet de définir le comportement à adopter lorsqu'un objet référencé est supprimé.

Dans le projet, plusieurs stratégies sont utilisées selon la nature de la relation :

— **CASCADE** : suppression des éléments dépendants lorsque leur existence dépend directement de l'objet principal ;
— **PROTECT** : empêche la suppression lorsqu'elle risque de compromettre l'intégrité des données ;
— **SET_NULL** : conserve l'enregistrement mais supprime la référence vers l'objet supprimé lorsque cette relation est facultative.

Le choix du comportement dépend donc de l'importance métier de chaque relation.

6.17 Migrations Django

Django utilise un système de migrations afin de gérer l'évolution de la structure de la base de données.

Lorsqu'une modification structurelle est apportée à un modèle, Django peut générer une migration décrivant les changements à appliquer.

Les principales commandes utilisées sont :

```
python manage.py makemigrations
python manage.py migrate
```

La commande `makemigrations` génère les migrations correspondant aux changements effectués dans les modèles.

La commande `migrate` applique ensuite ces migrations à la base de données.

Ce mécanisme facilite l'évolution progressive du schéma de la base tout au long du développement.

6.18 Gestion du périmètre multi-agences

La structure de la base de données joue également un rôle important dans la gestion des accès multi-agences.

Les entités principales sont associées à une agence lorsque cela est nécessaire.

Lorsqu'un utilisateur standard consulte des informations, l'application limite les requêtes aux données appartenant à son agence d'affectation.

Les profils disposant d'un accès global peuvent, quant à eux, consulter les informations provenant de plusieurs agences selon leurs autorisations.

Cette règle est appliquée au niveau de la logique serveur et ne repose pas uniquement sur les éléments affichés dans l'interface.

6.19 Cohérence entre base de données et logique métier

La base de données assure principalement la persistance et l'intégrité structurelle des informations.

Cependant, certaines règles nécessitent une logique métier supplémentaire dans l'application.

C'est notamment le cas de la règle concernant l'avance.

Pour déterminer si un dossier peut être traité, l'application peut vérifier :

— si le client est autorisé à poursuivre sans avance ;
— si le dossier ne nécessite pas d'avance ;
— ou si une avance a effectivement été enregistrée.

De la même manière, le montant payé et le reste à payer sont calculés à partir des opérations financières enregistrées.

La solution combine ainsi les contraintes de la base de données et la logique applicative afin d'assurer un fonctionnement cohérent.

6.20 Avantages de la conception adoptée

La structure retenue présente plusieurs avantages pour l'application :

— centralisation des informations métier dans une base relationnelle unique ;
— séparation claire des principales entités ;
— limitation de la duplication des données ;
— gestion cohérente des relations entre les modules ;
— prise en charge du fonctionnement multi-agences ;
— conservation de l'historique des opérations financières ;
— prise en charge de la traçabilité de certaines actions ;
— exploitation des données pour les tableaux de bord et les rapports ;
— possibilité de faire évoluer progressivement le modèle de données.

6.21 Conclusion

Ce chapitre a présenté la conception et l'organisation de la base de données du système de gestion interne de BMT Tourisme.

Le diagramme de classes a permis de visualiser les principales entités ainsi que les relations qui structurent le système. L'utilisation de MySQL, associée à l'ORM de Django, permet ensuite de traduire cette conception en une structure de données relationnelle exploitée par les différents modules de l'application.

La conception adoptée permet également d'intégrer les contraintes liées aux rôles, aux opérations financières, à la traçabilité et au fonctionnement multi-agences.

Le chapitre suivant sera consacré à **la réalisation et à la présentation des principaux modules de l'application**, afin de montrer concrètement la mise en œuvre des fonctionnalités définies au cours des phases précédentes.

Chapitre 7

Réalisation et Description des Modules

7.1 Introduction

Après les phases d'analyse, de spécification et de conception, le projet est passé à la phase de réalisation. Cette étape consiste à traduire les besoins fonctionnels et les choix de conception en une application web opérationnelle.

Le système développé pour BMT Tourisme est organisé autour d'une interface commune permettant aux différents employés d'accéder aux fonctionnalités correspondant à leurs rôles et à leurs permissions.

Ce chapitre présente les principales interfaces de l'application ainsi que le fonctionnement des modules réalisés.

7.2 Interface d'authentification

L'authentification constitue le point d'entrée de l'application. L'utilisateur doit disposer d'un compte valide avant de pouvoir accéder au système.

Après la saisie de ses informations de connexion, le système vérifie son identité puis détermine le profil employé associé au compte.

Une fois authentifié, l'utilisateur est redirigé vers son espace de travail.

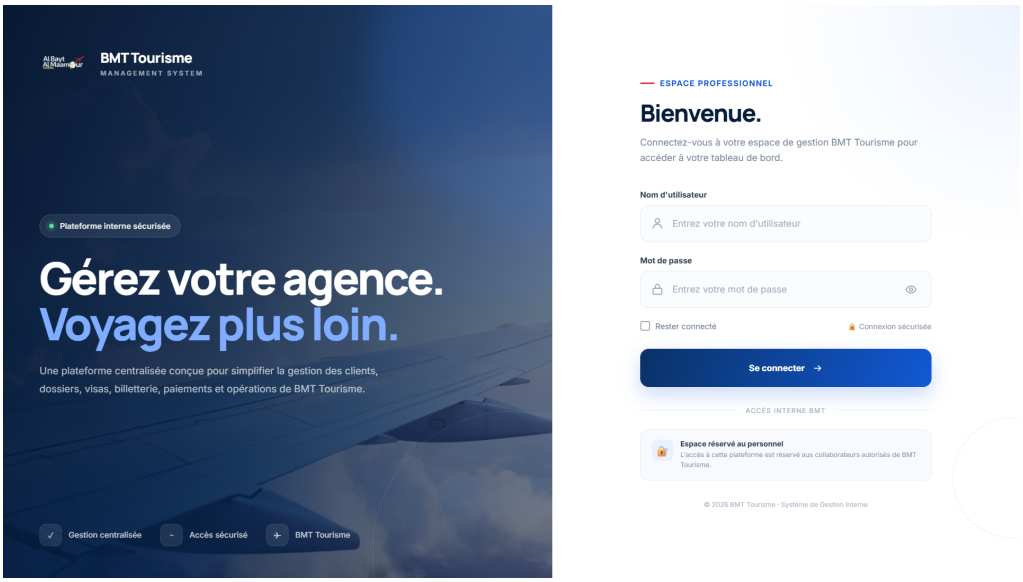


Figure 7.1 – Interface d'authentification

7.3 Interface générale de l'application

L'application repose sur une interface commune aux différents profils d'utilisateurs.

Cette interface comprend principalement une barre latérale de navigation, une zone supérieure contenant les informations utiles à l'utilisateur et une zone centrale consacrée au contenu du module sélectionné.

Les éléments du menu sont adaptés dynamiquement selon les permissions de l'utilisateur connecté. Ainsi, deux employés utilisant la même application peuvent disposer de fonctionnalités différentes.

Cette approche permet de conserver une interface cohérente tout en respectant les responsabilités de chaque profil.

7.4 Tableau de bord

Le tableau de bord constitue la page principale accessible après authentification.

Il fournit une vue synthétique de l'activité correspondant au périmètre de l'utilisateur connecté.

L'interface peut notamment présenter des indicateurs concernant les clients, les dossiers, les tâches ou d'autres informations pertinentes selon les droits disponibles.

Le tableau de bord constitue également un point d'accès rapide aux principales fonctionnalités de l'application.

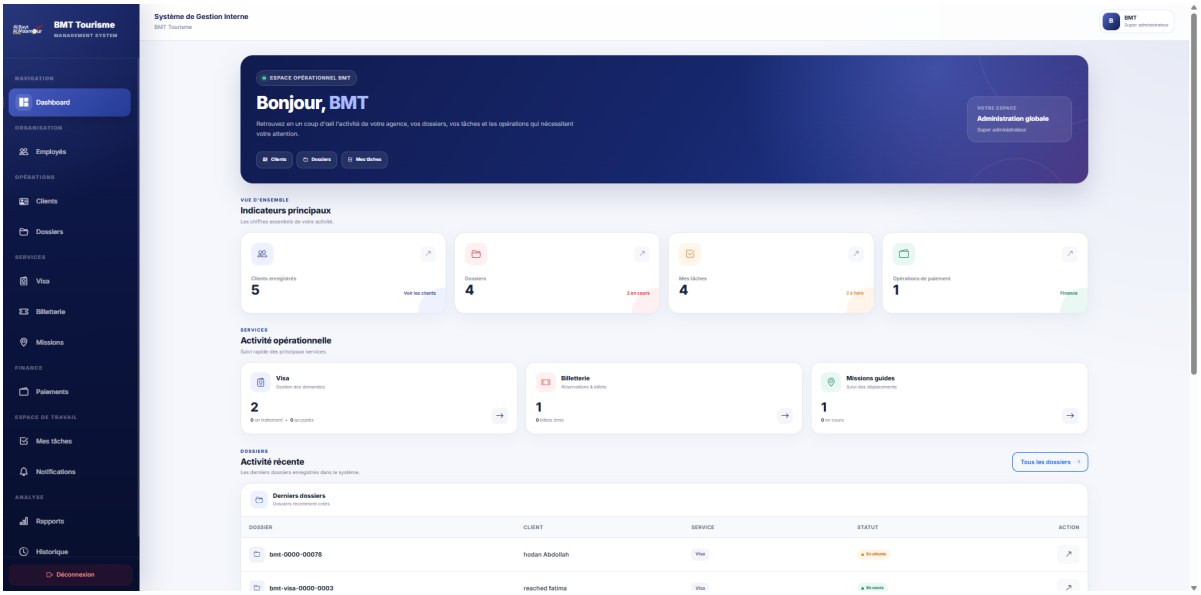


Figure 7.2 – Tableau de bord de l'application

7.5 Module de gestion des clients

Le module **Clients** permet de centraliser les informations relatives aux clients pris en charge par l'agence.

L'utilisateur disposant des permissions nécessaires peut consulter la liste des clients accessibles dans son périmètre.

L'interface facilite également la recherche afin de retrouver rapidement un client existant.

7.5.1 Liste des clients

La liste présente les principales informations permettant d'identifier les clients et d'accéder aux actions disponibles.

Les données affichées sont filtrées selon le périmètre de l'agence lorsque l'utilisateur ne dispose pas d'un accès global.

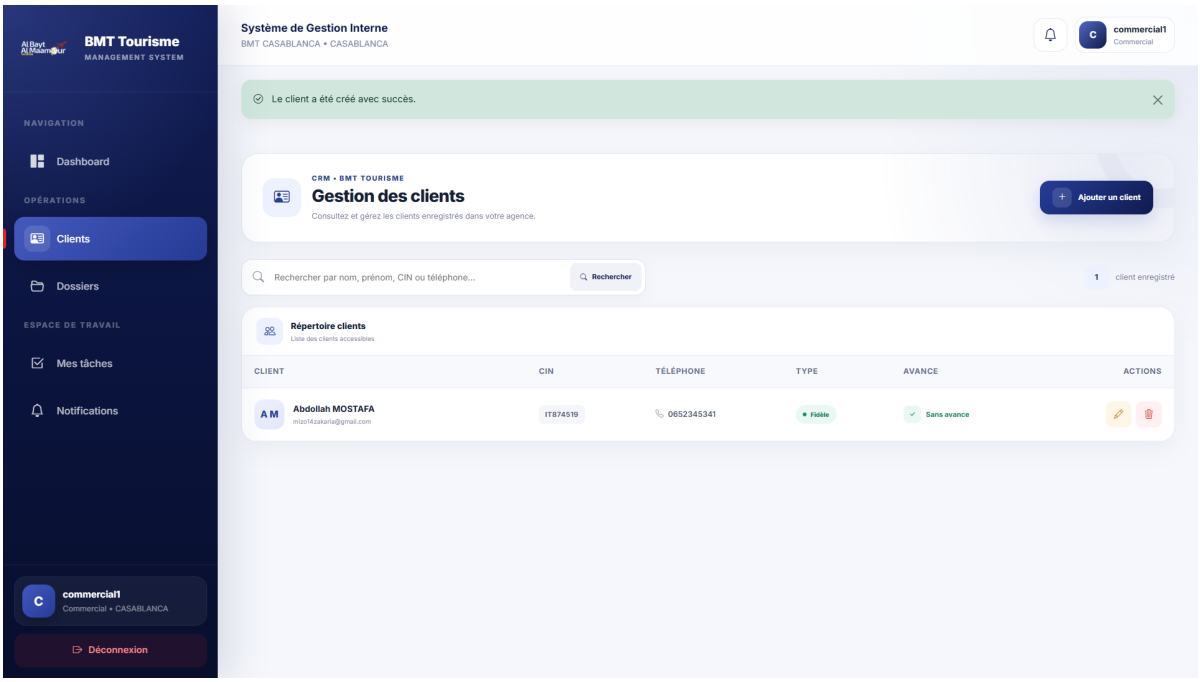


Figure 7.3 – Interface de gestion des clients

7.5.2 Ajout et modification d'un client

Un formulaire permet d'enregistrer les informations nécessaires à la création d'un client.

L'utilisateur peut notamment renseigner les informations d'identification et de contact ainsi que les paramètres utiles à son traitement.

L'agence du client est contrôlée selon les droits de l'utilisateur afin d'éviter l'enregistrement de données dans un périmètre non autorisé.

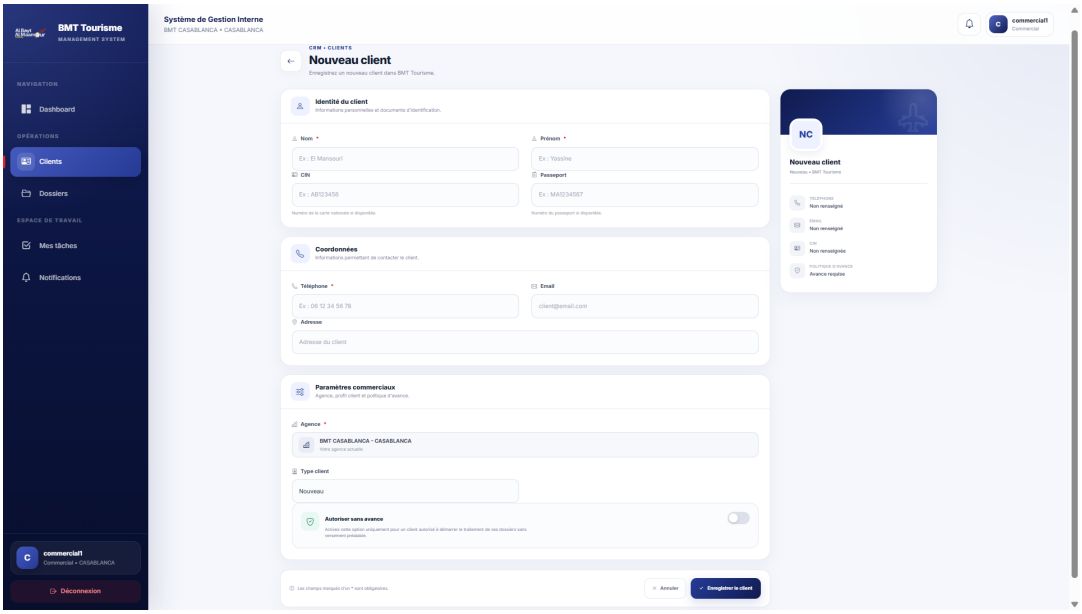


Figure 7.4 – Formulaire de gestion d'un client

7.6 Module de gestion des dossiers

Le module **Dossiers** représente une partie centrale de l'application.

Chaque dossier est associé à un client et permet de regrouper les informations liées au traitement de sa demande.

7.6.1 Liste des dossiers

L'interface de consultation permet d'obtenir une vue générale des dossiers accessibles à l'utilisateur.

Les informations principales permettent notamment d'identifier le client, le service concerné et l'état du dossier.

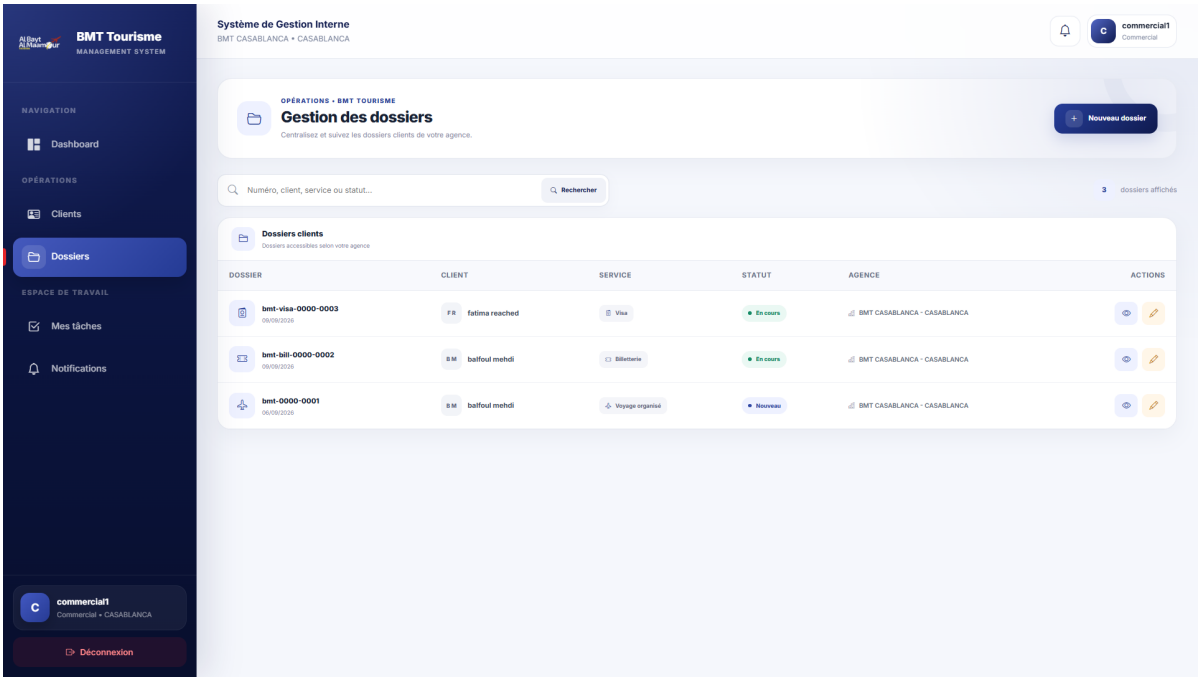


Figure 7.5 – Liste des dossiers

7.6.2 Création d'un dossier

Le formulaire de création permet de sélectionner le client concerné, le type de service, le montant et les autres informations nécessaires.

Les règles d'accès sont appliquées afin d'empêcher un utilisateur standard de créer un dossier dans une agence différente de son agence d'affectation.

7.6.3 Détail d'un dossier

La page de détail permet de regrouper les informations relatives à un dossier dans une même interface.

Elle constitue un point central permettant de consulter les données du client et les différentes opérations associées au traitement du dossier.

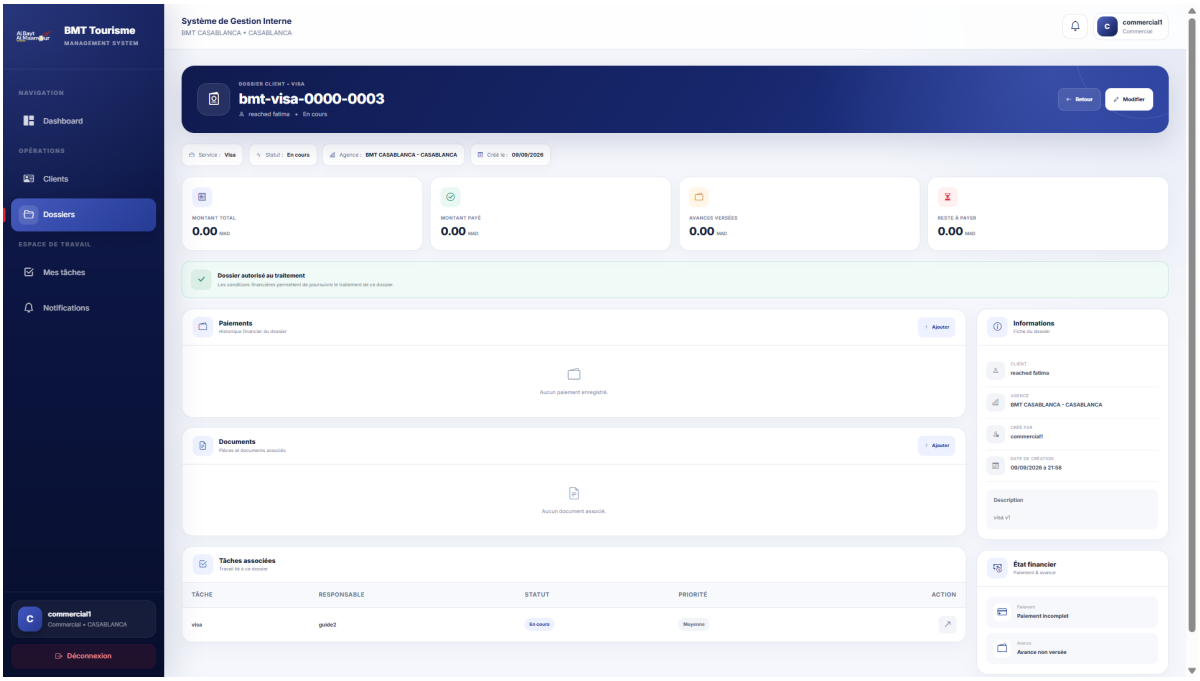


Figure 7.6 – Détail d'un dossier

7.7 Module de gestion des tâches

Le module **Tâches** permet d'organiser les opérations à réaliser dans le cadre du traitement des dossiers.

Une tâche peut être attribuée à un employé et dispose notamment d'un statut et d'un niveau de priorité.

Les utilisateurs autorisés peuvent créer ou modifier les tâches, tandis que les employés concernés peuvent suivre les opérations qui leur sont affectées selon leurs permissions.

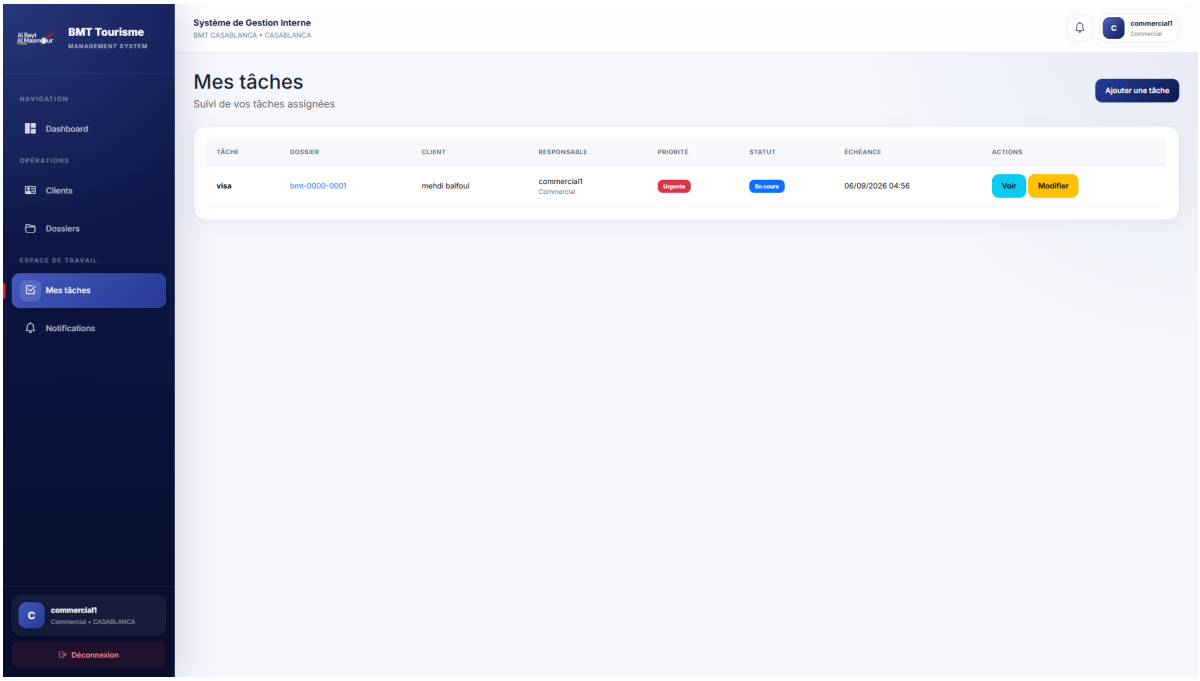


Figure 7.7 – Interface de gestion des tâches

7.8 Module de gestion des paiements

Le module **Paiements** permet d'enregistrer et de suivre les opérations financières associées aux dossiers.

Il prend en charge les avances, les paiements, les règlements de solde ainsi que les remboursements.

L'application exploite ensuite ces opérations afin de calculer le montant net payé et le montant restant à régler.

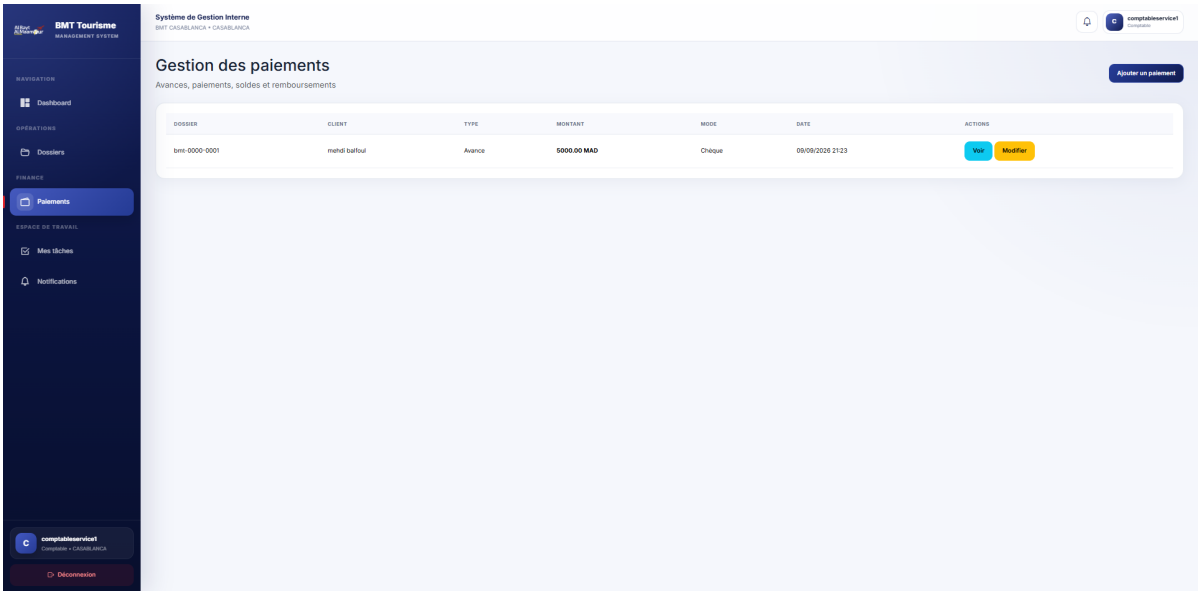


Figure 7.8 – Gestion des paiements

7.9 Module Visa

Le module **Visa** permet aux utilisateurs autorisés de suivre les demandes associées aux dossiers concernés.

L'interface permet de conserver les informations relatives à la destination, au type de visa, aux rendez-vous, au dépôt et au résultat de la demande.

Le statut facilite le suivi de la progression de chaque demande depuis la préparation des documents jusqu'à la décision finale.

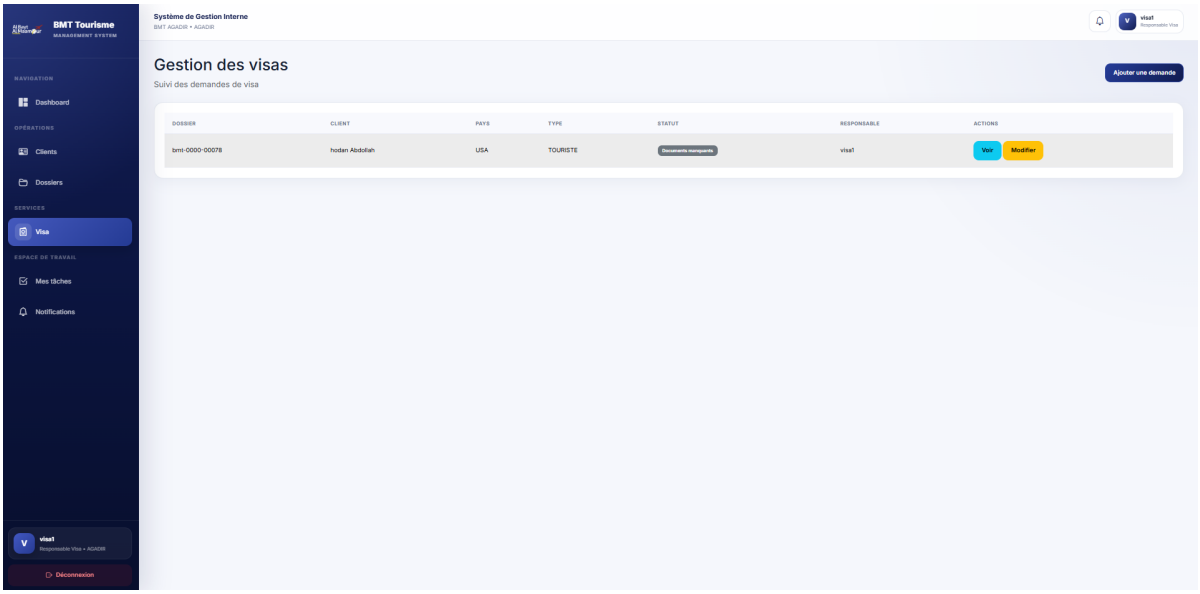


Figure 7.9 – Interface de suivi des demandes de visa

7.10 Module Billetterie

Le module **Billetterie** est consacré à la gestion des réservations et des billets.

Il permet d'enregistrer les informations relatives au trajet, à la compagnie, aux dates, à la référence de réservation, au prix et au statut du billet.

Plusieurs billets peuvent être associés à un même dossier lorsque la prestation le nécessite.

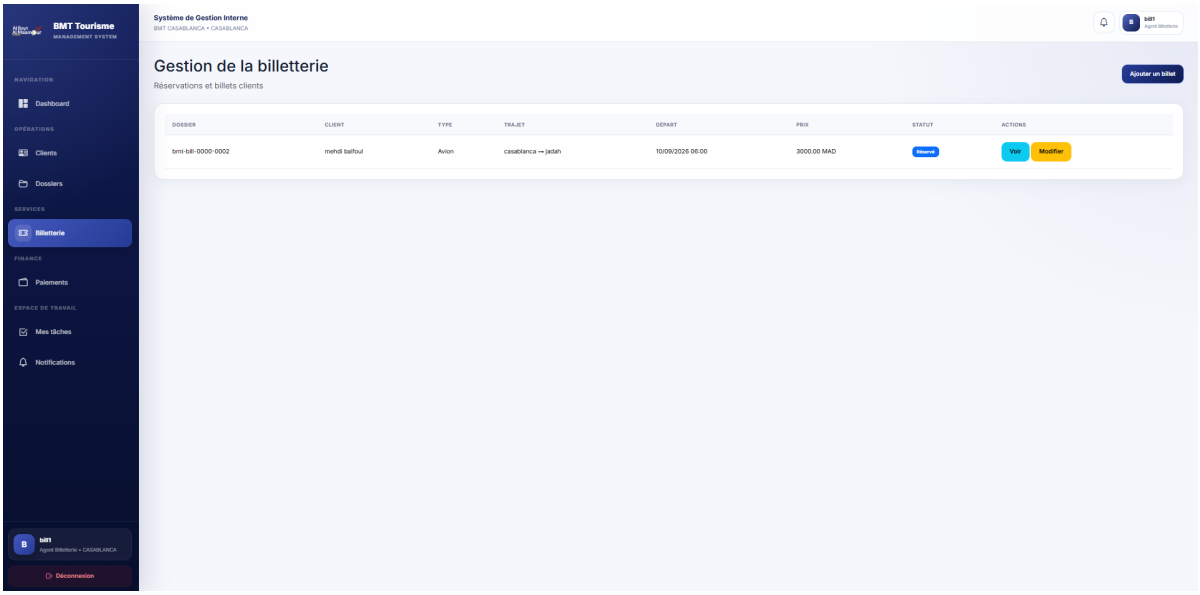


Figure 7.10 – Interface de gestion de la billetterie

7.11 Module Guides et Missions

Le module **Guides** permet de gérer les profils des guides ainsi que les missions qui leur sont attribuées.

Une mission est associée à un dossier et contient les informations nécessaires à son organisation, notamment le lieu, les dates et son état.

Le guide peut accéder aux missions qui le concernent et effectuer les actions correspondant à ses permissions.

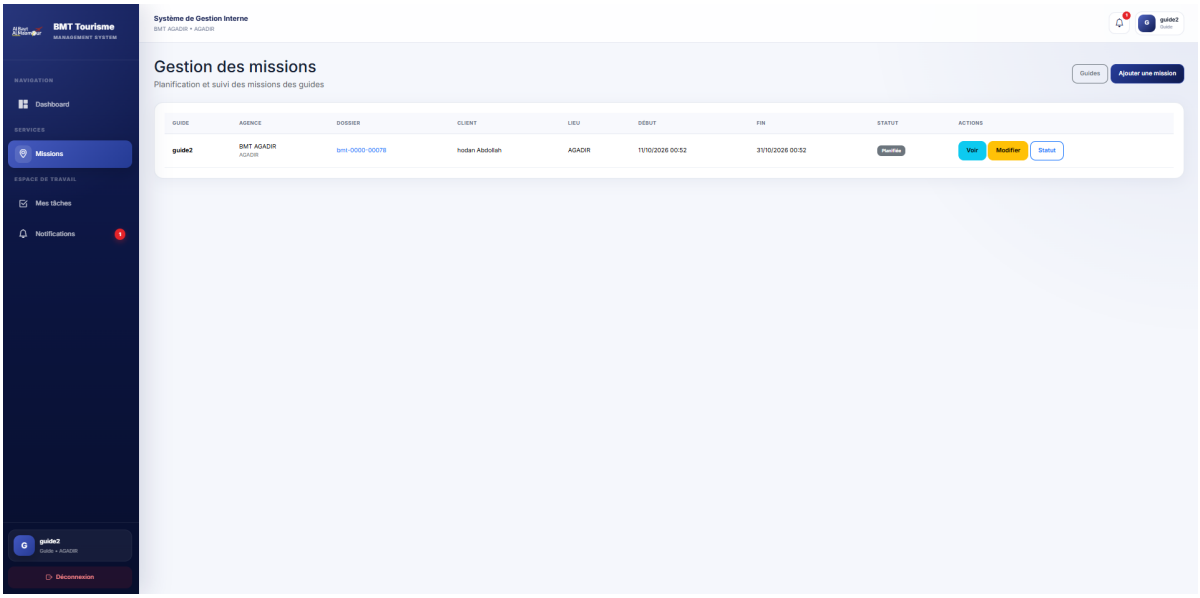


Figure 7.11 – Gestion des missions des guides

7.12 Module de gestion des employés

La gestion des employés permet d'administrer les profils internes utilisant l'application.

Chaque employé peut être associé à un compte utilisateur, à une agence, à un rôle et à des permissions spécifiques.

Cette organisation permet d'adapter précisément les accès aux responsabilités de chaque utilisateur.

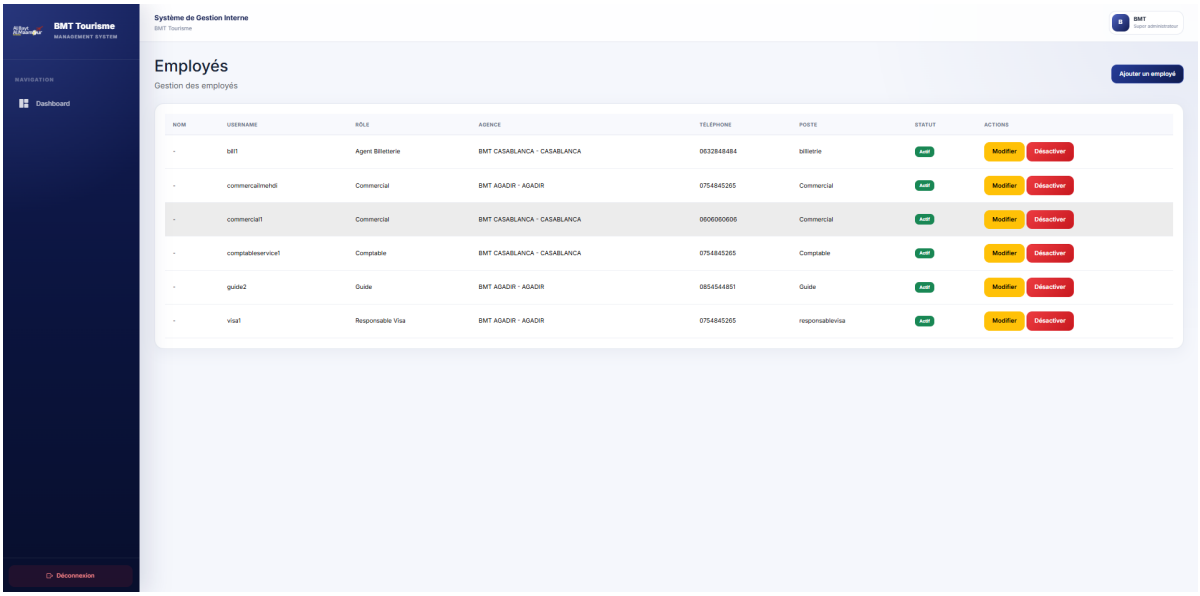


Figure 7.12 – Interface de gestion des employés

7.13 Module de gestion des agences

L'application utilise une structure multi-agences pour associer les employés et les dossiers à leur agence respective.

La gestion administrative des agences constitue une fonctionnalité prévue pour une évolution future du système.

7.14 Notifications

L'application dispose d'un mécanisme permettant de transmettre des notifications aux employés.

Chaque utilisateur peut ainsi consulter les informations qui lui sont destinées et identifier les notifications déjà lues ou nécessitant encore son attention.

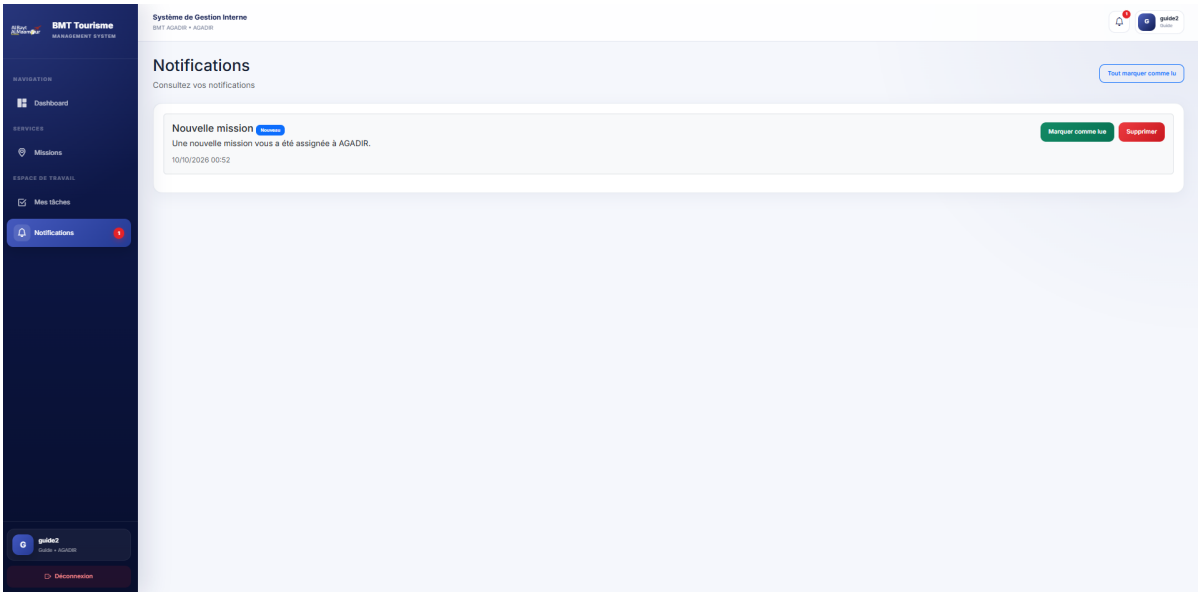


Figure 7.13 – Interface des notifications

7.15 Historique des actions

Certaines opérations importantes réalisées dans l'application peuvent être enregistrées dans un historique.

Cette fonctionnalité améliore la traçabilité en permettant d'identifier l'utilisateur concerné, l'action effectuée, le module correspondant ainsi que la date de l'opération.

7.16 Rapports et indicateurs

Le module **Rapports** exploite les données enregistrées dans les différents modules afin de fournir une vision synthétique de l'activité.

Les indicateurs peuvent concerner les dossiers, les clients, les opérations financières, les visas, la billetterie ou encore les missions.

Pour les utilisateurs standards, les résultats sont limités au périmètre autorisé. Les profils disposant d'un accès global peuvent consulter des informations couvrant plusieurs agences.

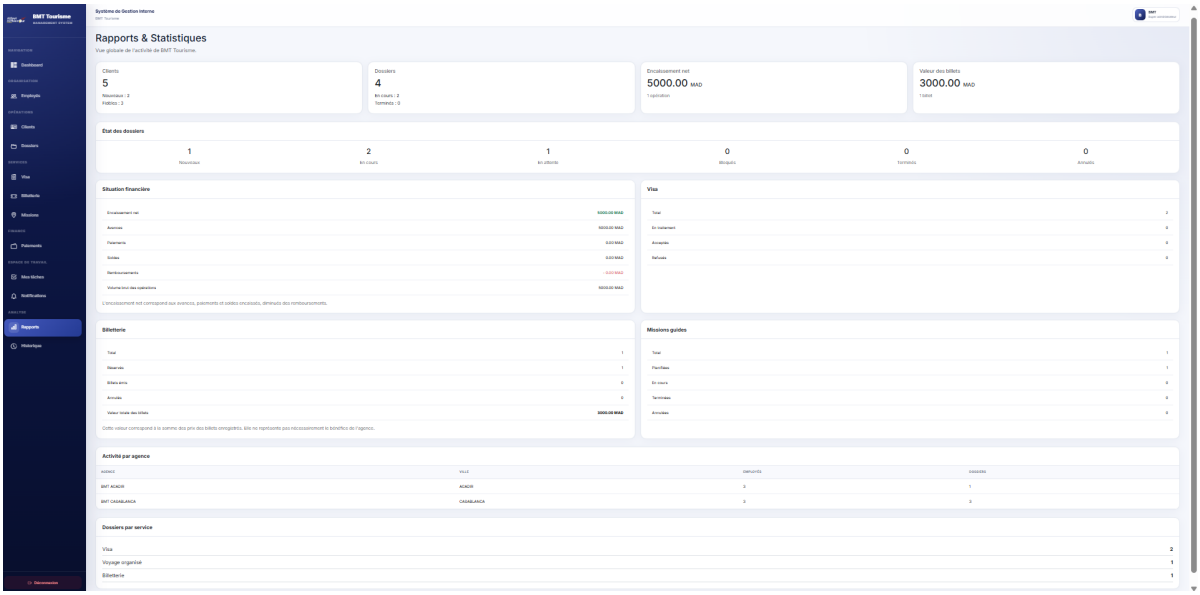


Figure 7.14 – Interface des rapports et indicateurs

7.17 Cohérence de l'interface utilisateur

Les différentes interfaces ont été conçues autour d'une identité visuelle commune afin d'assurer une navigation cohérente dans l'ensemble de l'application.

La barre latérale constitue l'élément principal de navigation, tandis que les tableaux, formulaires, indicateurs et éléments de statut suivent une présentation homogène.

Une attention particulière a également été portée à la lisibilité des informations, à la hiérarchisation visuelle et à l'adaptation de l'interface aux différents profils d'utilisateurs.

L'objectif est de proposer un environnement de travail professionnel et adapté à une utilisation quotidienne.

7.18 Conclusion

Ce chapitre a présenté la réalisation des principaux modules du système de gestion interne de BMT Tourisme.

L'application centralise les différentes opérations métier dans une interface commune tout en adaptant les fonctionnalités aux rôles, aux permissions et au périmètre organisationnel de chaque utilisateur.

Les différents modules permettent ainsi de couvrir le cycle de traitement des dossiers, depuis la gestion du client jusqu'au suivi des prestations, des paiements et des opérations internes.

Le chapitre suivant sera consacré à **la sécurité et à la gestion des accès**, afin de présenter plus précisément les mécanismes utilisés pour protéger les fonctionnalités et les données de l'application.

Chapitre 8

Sécurité et Gestion des Accès

8.1 Introduction

Dans une application de gestion interne, la sécurité constitue un élément essentiel, car le système manipule des informations relatives aux clients, aux employés, aux dossiers ainsi qu'aux opérations financières et administratives.

La protection de ces informations nécessite non seulement l'authentification des utilisateurs, mais également un contrôle précis des fonctionnalités et des données auxquelles chaque employé peut accéder.

Dans le système développé pour BMT Tourisme, la sécurité repose sur plusieurs mécanismes complémentaires : l'authentification Django, la gestion des rôles et des permissions métier, le contrôle du périmètre des agences, la protection des requêtes et la vérification des autorisations côté serveur.

8.2 Authentification des utilisateurs

L'accès à l'application nécessite une authentification préalable.

Le système utilise le mécanisme d'authentification fourni par Django. Chaque utilisateur possède un compte permettant de vérifier son identité avant de lui donner accès aux fonctionnalités internes.

Une fois connecté, le compte utilisateur est associé à un profil **Employe**, qui contient les informations professionnelles nécessaires au fonctionnement de l'application.

Cette séparation permet de distinguer les informations techniques d'authentification des informations métier de l'employé.

8.3 Protection des pages authentifiées

Les pages internes de l'application ne doivent pas être accessibles à un utilisateur non connecté.

Pour cette raison, les vues nécessitant une authentification sont protégées au niveau

du serveur.

Django fournit notamment le décorateur :

```
@login_required
```

Ce mécanisme vérifie que l'utilisateur dispose d'une session authentifiée avant d'autoriser l'exécution de la vue.

Dans le cas contraire, l'utilisateur est redirigé vers l'interface de connexion.

8.4 Gestion des rôles

Chaque employé peut être associé à un rôle représentant sa fonction principale au sein de l'organisation.

Le système prend notamment en considération les profils suivants :

— Administrateur ;
— Directeur ;
— Commercial ;
— Responsable Visa ;
— Agent Billetterie ;
— Comptable ;
— Guide ;
— autres profils employés selon les besoins.

Le rôle constitue un premier niveau d'organisation des droits, mais il n'est pas utilisé comme unique mécanisme de contrôle.

Afin d'obtenir une gestion plus flexible, le système repose également sur des permissions métier.

8.5 Permissions métier

Les permissions métier permettent de définir précisément les opérations qu'un utilisateur est autorisé à réaliser.

Au lieu de limiter l'application à des règles fixes basées uniquement sur le nom du rôle, chaque rôle peut regrouper plusieurs permissions.

Par exemple, certaines permissions permettent de :

— consulter les clients ;
— créer ou modifier un client ;
— consulter ou créer des dossiers ;
— gérer les documents ;
— consulter, créer ou affecter des tâches ;
— consulter ou gérer les paiements ;
— gérer les demandes de visa ;
— gérer la billetterie ;
— consulter ou gérer les missions ;
— gérer les employés ;
— gérer les agences ;
— consulter les rapports.

Cette approche permet d'adapter les droits aux responsabilités réelles des utilisateurs.

8.6 Permissions spécifiques aux employés

En complément des permissions héritées du rôle, l'application permet d'attribuer directement des permissions à un employé.

Le contrôle d'une autorisation prend donc en considération :

1. les permissions directement attribuées à l'employé ;
2. les permissions associées à son rôle.

Cette organisation permet, par exemple, d'accorder exceptionnellement une fonctionnalité supplémentaire à un employé sans modifier les droits de tous les utilisateurs possédant le même rôle.

8.7 Contrôle des permissions côté serveur

Les permissions ne sont pas vérifiées uniquement dans l'interface utilisateur.

Un mécanisme spécifique de contrôle des permissions métier est utilisé au niveau des vues Django.

Le principe général du contrôle peut être représenté ainsi :

Utilisateur ⟶ **Authentification** ⟶ **Profil Employé** ⟶ **Permission** ⟶

Accès ou Refus

Lorsqu'un utilisateur tente d'accéder à une fonctionnalité protégée, le système vérifie d'abord son authentification, puis son profil employé et enfin la permission nécessaire.

Si la permission n'est pas disponible, l'opération est refusée.

8.8 Décorateur de permissions métier

Afin de centraliser la vérification des autorisations, un décorateur personnalisé est utilisé dans l'application.

Son utilisation dans une vue peut suivre le principe suivant :

```
@login_required
@permission_required_metier('payments.manage')
def payment_create(request):
    ...
```

Avant d'exécuter la vue, le décorateur vérifie si l'utilisateur possède la permission demandée.

Cette approche évite de répéter la même logique de vérification dans chaque fonctionnalité et facilite la maintenance des règles d'accès.

8.9 Gestion des utilisateurs inactifs

Le contrôle d'accès tient également compte de l'état du profil employé.

Un employé marqué comme inactif ne doit pas pouvoir exploiter ses permissions métier même si son rôle contient encore des autorisations.

Cette vérification constitue une protection supplémentaire permettant de désactiver l'accès métier d'un employé sans devoir modifier l'ensemble de la configuration de son rôle.

8.10 Contrôle d'accès multi-agences

La gestion des permissions fonctionnelles ne suffit pas à elle seule pour protéger les données.

Un utilisateur peut, par exemple, avoir le droit de consulter les dossiers sans pour autant être autorisé à consulter les dossiers de toutes les agences.

Le système distingue donc deux notions :

— **ce que l'utilisateur peut faire** : contrôlé par les permissions ;
— **sur quelles données il peut le faire** : contrôlé par le périmètre de l'agence.

Cette séparation constitue un principe important dans l'architecture de sécurité de l'application.

8.11 Accès global et accès local

Le système distingue les utilisateurs disposant d'un accès global et ceux dont l'accès est limité à une agence.

Les profils disposant d'un accès global peuvent consulter les données de plusieurs agences selon leurs responsabilités.

Les autres employés travaillent uniquement sur les informations correspondant à leur agence d'affectation.

Le principe peut être résumé comme suit :

Profil	Périmètre
Superutilisateur	Accès global au système.
Administrateur	Accès global aux différentes agences selon ses permissions.
Directeur	Accès global aux différentes agences selon ses permissions.
Autres employés	Accès limité principalement aux données de leur agence d'affectation.

Table 8.1 – Principe de gestion du périmètre des agences

8.12 Filtrage des données côté serveur

Le filtrage par agence est effectué dans la logique serveur.

Lorsqu'un utilisateur standard consulte une liste de dossiers, de clients ou d'autres données métier, la requête est limitée aux éléments appartenant à son agence.

Cette protection est importante, car masquer simplement les données dans l'interface ne serait pas suffisant.

Un utilisateur ne doit pas pouvoir contourner les restrictions en modifiant manuellement une URL ou les paramètres d'une requête.

8.13 Protection contre les accès directs

Les pages de détail, de modification et les autres opérations utilisant un identifiant doivent également vérifier que l'objet demandé appartient au périmètre autorisé.

Ainsi, même si un utilisateur connaît l'identifiant d'un dossier appartenant à une autre agence, l'application ne doit pas lui permettre d'y accéder lorsque son profil ne dispose pas des droits nécessaires.

La sécurité est donc appliquée au niveau des données récupérées par le serveur et pas uniquement au niveau de la navigation.

8.14 Validation des données envoyées

Les contrôles d'accès doivent également être appliqués lors de l'enregistrement des données.

Par exemple, un utilisateur limité à une agence ne doit pas pouvoir modifier manuellement une requête afin d'associer un dossier, un client ou une autre opération à une agence non autorisée.

Les données reçues par le serveur sont donc contrôlées avant leur enregistrement afin de respecter les règles métier et le périmètre de l'utilisateur.

8.15 Protection des opérations sensibles

Certaines actions modifient directement l'état des données, par exemple une suppression ou un changement d'état.

Pour ce type d'opérations, l'application privilégie des requêtes adaptées à la modification des données plutôt qu'une simple navigation par URL.

Les contrôles de permissions et de périmètre restent appliqués avant l'exécution de l'opération.

Cette organisation réduit les risques de déclenchement involontaire d'une action sensible.

8.16 Protection CSRF

Django fournit un mécanisme de protection contre les attaques de type **CSRF (Cross-Site Request Forgery)**.

Dans les formulaires utilisant la méthode POST, un jeton CSRF est intégré à l'aide de :

```
{% csrf_token %}
```

Ce jeton permet au serveur de vérifier que la requête provient d'un formulaire généré dans le contexte de l'application.

Il contribue ainsi à limiter l'exécution de requêtes POST frauduleuses envoyées depuis une source externe.

8.17 Sécurité au niveau de l'interface

L'interface utilise également les permissions afin de déterminer les éléments de navigation et les actions qui doivent être présentés à l'utilisateur.

Par exemple, un bouton permettant de gérer les paiements peut être masqué pour un utilisateur ne disposant pas de la permission correspondante.

Cependant, cette adaptation de l'interface est principalement destinée à améliorer l'expérience utilisateur.

Elle ne remplace jamais les contrôles réalisés côté serveur.

Le principe adopté peut donc être résumé ainsi :

Interface adaptée + **Contrôle serveur** = **Gestion cohérente des accès**

8.18 Traçabilité des actions

La sécurité d'un système de gestion ne concerne pas uniquement la prévention des accès non autorisés.

La traçabilité permet également de conserver des informations sur certaines actions importantes réalisées dans l'application.

Le modèle **HistoriqueAction** permet notamment d'enregistrer :

— l'employé concerné ;

— l'action effectuée ;
— le module concerné ;
— l'objet éventuellement concerné ;
— une description ;
— la date de l'action.

Ces informations facilitent le suivi des opérations réalisées dans le système.

8.19 Séparation entre interface et sécurité réelle

Un principe important appliqué durant le développement consiste à ne pas considérer l'interface comme une barrière de sécurité.

Le fait de masquer un bouton ou une entrée du menu ne garantit pas qu'un utilisateur ne tentera pas d'accéder directement à l'URL correspondante.

Pour cette raison, les restrictions sont appliquées à plusieurs niveaux :

1. contrôle de l'authentification ;
2. contrôle des permissions métier ;
3. contrôle du périmètre de l'agence ;
4. validation des données envoyées ;
5. protection des opérations sensibles ;
6. adaptation de l'interface.

Cette approche permet d'obtenir une gestion des accès plus robuste qu'une protection reposant uniquement sur l'affichage.

8.20 Synthèse de la stratégie de contrôle d'accès

La stratégie adoptée peut être représentée par la chaîne de vérification suivante :

Requête utilisateur

↓

Utilisateur authentifié ?

↓

Profil employé actif ?

↓

Permission nécessaire disponible ?

↓

Objet dans le périmètre autorisé ?

↓

Validation des données

↓

Exécution de l'opération

Cette succession de contrôles permet de combiner la gestion fonctionnelle des autorisations avec la séparation organisationnelle des données.

8.21 Conclusion

Ce chapitre a présenté les principaux mécanismes utilisés pour sécuriser l'accès au système de gestion interne de BMT Tourisme.

L'authentification Django permet d'identifier les utilisateurs, tandis que les rôles et les permissions métier déterminent les fonctionnalités accessibles.

Le contrôle du périmètre des agences complète cette approche en limitant les données sur lesquelles un utilisateur peut agir.

Enfin, les vérifications côté serveur, la protection CSRF, la validation des requêtes et la traçabilité des actions permettent de renforcer la cohérence de la stratégie de sécurité mise en place.

Le chapitre suivant sera consacré aux **tests et à la validation de l'application**, afin de vérifier le comportement des principales fonctionnalités et le respect des règles métier définies.

Chapitre 9

Tests et Validation

9.1 Introduction

La phase de test constitue une étape essentielle dans le cycle de développement d'une application. Elle permet de vérifier que les fonctionnalités réalisées correspondent aux besoins définis et que les principales règles métier sont correctement respectées.

Dans le cadre du système de gestion interne de BMT Tourisme, la stratégie de validation porte principalement sur l'authentification, les permissions, le périmètre multi-agences, la gestion des clients et des dossiers ainsi que sur les différents modules métier de l'application.

Ce chapitre présente la démarche de test retenue et les principaux scénarios permettant de valider le comportement du système.

9.2 Objectifs des tests

Les tests ont pour objectif de vérifier plusieurs aspects du système :

— le bon fonctionnement des principales fonctionnalités ;
— le respect des règles métier ;
— le contrôle des permissions ;
— la séparation des données entre les agences ;
— la cohérence des données enregistrées ;
— la gestion correcte des opérations financières ;
— la protection des fonctionnalités sensibles ;
— le comportement général des interfaces.

La validation ne concerne donc pas uniquement l'affichage des pages, mais également les règles appliquées côté serveur.

9.3 Méthodologie de validation

La validation de l'application repose sur des scénarios représentant des situations proches de son utilisation réelle.

Pour chaque scénario, plusieurs éléments doivent être vérifiés :

1. les conditions initiales ;
2. l'action réalisée par l'utilisateur ;
3. le comportement attendu ;
4. le résultat obtenu lors de l'exécution du test.

Cette méthode permet de vérifier progressivement les différents modules et de détecter les éventuelles incohérences avant la validation finale de l'application.

9.4 Test de l'authentification

L'authentification constitue le premier niveau de contrôle du système.

Les scénarios suivants permettent de vérifier son fonctionnement.

ID	Scénario	Résultat attendu	Résultat
T01	Connexion avec des identifiants valides.	L'utilisateur est authentifié et accède à son espace de travail.	À valider
T02	Connexion avec des identifiants incorrects.	L'accès est refusé et l'utilisateur reste sur l'interface d'authentification.	À valider
T03	Tentative d'accès à une page protégée sans authentification.	L'utilisateur est redirigé vers la page de connexion.	À valider

Table 9.1 – Scénarios de test de l'authentification

9.5 Test des rôles et permissions

Cette série de tests permet de vérifier qu'un utilisateur ne peut exécuter que les opérations correspondant à ses droits.

ID	Scénario	Résultat attendu	Résultat
T04	Accès à une fonctionnalité pour laquelle l'utilisateur possède la permission nécessaire.	L'accès est autorisé.	À valider
T05	Accès à une fonctionnalité sans la permission nécessaire.	L'opération est refusée par le serveur.	À valider
T06	Attribution d'une permission spécifique directement à un employé.	L'employé peut accéder à la fonctionnalité correspondante sans modifier les droits des autres utilisateurs de son rôle.	À valider
T07	Tentative d'utilisation des permissions métier par un profil employé désactivé.	L'accès métier est refusé.	À valider

Table 9.2 – Scénarios de test des rôles et permissions

9.6 Test du contrôle multi-agences

La séparation des données entre les agences constitue une règle importante du système.

Les tests doivent vérifier non seulement l'affichage des données, mais également la protection contre les accès directs.

ID	Scénario	Résultat attendu	Résultat
T08	Un employé standard consulte les dossiers.	Seuls les dossiers appartenant à son périmètre autorisé sont affichés.	À valider
T09	Un employé tente d'accéder directement à un dossier d'une autre agence.	L'accès est refusé.	À valider
T10	Un employé tente d'envoyer manuellement l'identifiant d'une autre agence dans un formulaire.	Le serveur refuse ou neutralise l'association non autorisée.	À valider
T11	Un utilisateur disposant d'un accès global consulte les données.	Les informations des agences correspondant à son périmètre global sont accessibles.	À valider

Table 9.3 – Scénarios de test du contrôle multi-agences

9.7 Test du module Clients

Les tests du module Clients doivent vérifier les opérations principales de consultation et de gestion des clients.

ID	Scénario	Résultat attendu	Résultat
T12	Création d'un client avec des informations valides.	Le client est enregistré dans la base de données.	À valider
T13	Modification d'un client existant.	Les nouvelles informations sont correctement enregistrées.	À valider
T14	Tentative d'enregistrement d'une CIN déjà utilisée.	Le doublon est refusé conformément aux contraintes définies.	À valider
T15	Consultation de la liste des clients par un utilisateur local.	La liste respecte le périmètre de son agence.	À valider

Table 9.4 – Scénarios de test du module Clients

9.8 Test du module Dossiers

Le dossier étant un élément central du système, plusieurs scénarios doivent être vérifiés.

ID	Scénario	Résultat attendu	Résultat
T16	Création d'un dossier valide pour un client accessible.	Le dossier est créé et associé au client et à l'agence correspondants.	À valider
T17	Création d'un dossier avec un numéro déjà existant.	La création est refusée afin de préserver l'unicité du numéro de dossier.	À valider
T18	Consultation du détail d'un dossier autorisé.	Les informations et opérations associées au dossier sont accessibles selon les permissions.	À valider
T19	Tentative d'accès à un dossier situé hors du périmètre autorisé.	Le serveur refuse l'accès.	À valider

Table 9.5 – Scénarios de test du module Dossiers

9.9 Test de la règle d'avance

La gestion de l'avance constitue une règle métier importante dans le traitement des dossiers.

Plusieurs situations doivent être vérifiées.

ID	Scénario	Résultat attendu	Résultat
T20	Dossier nécessitant une avance sans avance enregistrée.	Le dossier ne satisfait pas encore la condition nécessaire au traitement.	À valider
T21	Enregistrement d'une avance pour le dossier.	Le système détecte qu'une avance a été versée.	À valider
T22	Client autorisé à poursuivre sans avance.	Le dossier peut satisfaire la règle de traitement même sans avance enregistrée.	À valider
T23	Dossier configuré comme ne nécessitant pas d'avance.	Le traitement peut se poursuivre sans versement préalable.	À valider

Table 9.6 – Scénarios de validation de la règle d'avance

9.10 Test du module Paiements

Les opérations financières nécessitent une attention particulière afin de vérifier la cohérence des montants calculés.

ID	Scénario	Résultat attendu	Résultat
T24	Enregistrement d'une avance.	L'opération est associée au dossier et prise en compte dans les calculs.	À valider
T25	Ajout de plusieurs paiements.	Le montant net payé correspond à la somme des opérations positives enregistrées.	À valider
T26	Enregistrement d'un remboursement.	Le remboursement est déduit du montant net payé.	À valider
T27	Paiement complet du montant du dossier.	Le reste à payer devient nul et le dossier est considéré comme soldé selon la règle définie.	À valider

Table 9.7 – Scénarios de test des paiements

9.11 Test du module Tâches

Les tests du module Tâches doivent notamment vérifier l'affectation et le contrôle des autorisations.

ID	Scénario	Résultat attendu	Résultat
T28	Création d'une tâche par un utilisateur autorisé.	La tâche est correctement enregistrée.	À valider
T29	Affectation d'une tâche à un employé autorisé.	La tâche est associée au responsable sélectionné.	À valider
T30	Modification du statut d'une tâche.	Le nouveau statut est correctement enregistré.	À valider
T31	Tentative d'affectation hors du périmètre autorisé.	L'opération est refusée.	À valider

Table 9.8 – Scénarios de test du module Tâches

9.12 Test des modules Visa, Billetterie et Missions

Les modules spécialisés doivent également respecter les permissions et le périmètre des agences.

ID	Scénario	Résultat attendu	Résultat
T32	Création ou modification d'une demande de visa par un utilisateur autorisé.	Les informations du visa sont correctement enregistrées.	À valider
T33	Création d'un billet associé à un dossier accessible.	Le billet est enregistré et associé au dossier correspondant.	À valider
T34	Création d'une mission pour un guide.	La mission est associée au guide et au dossier sélectionnés.	À valider
T35	Un guide consulte ses missions selon ses droits.	Seules les informations autorisées lui sont accessibles.	À valider

Table 9.9 – Tests des modules spécialisés

9.13 Test des notifications et de l'historique

Les mécanismes de notification et de traçabilité doivent être vérifiés afin de s'assurer que les informations sont associées aux bons utilisateurs et aux bonnes opérations.

Les principaux points à contrôler sont :

— l'affichage des notifications destinées à l'employé ;
— la modification de leur état de lecture ;
— l'enregistrement des actions prévues dans l'historique ;
— l'identification de l'utilisateur ayant effectué l'action ;
— la conservation de la date et du module concernés.

9.14 Test des rapports

Le module de rapports exploite plusieurs sources de données.

Sa validation doit notamment vérifier que :

— les indicateurs correspondent aux données enregistrées ;
— les remboursements sont correctement pris en compte dans les calculs financiers ;
— les informations sont filtrées selon le périmètre de l'utilisateur ;
— les utilisateurs disposant d'un accès global obtiennent les informations correspondant à ce périmètre.

9.15 Validation de l'interface utilisateur

Après la finalisation de l'interface graphique, une vérification visuelle et fonctionnelle doit également être effectuée.

Cette validation porte notamment sur :

— la cohérence du menu de navigation ;
— l'affichage des actions selon les permissions ;
— la lisibilité des tableaux et formulaires ;
— l'affichage des messages d'erreur et de confirmation ;
— la cohérence des statuts et indicateurs visuels ;
— l'adaptation de l'interface à différentes tailles d'écran ;
— la cohérence générale de l'identité visuelle.

9.16 Bilan de validation

Les scénarios présentés dans ce chapitre constituent la base de validation fonctionnelle et métier de l'application.

À l'issue de la phase de test finale, le résultat de chaque scénario pourra être renseigné afin de distinguer les tests validés des éventuelles anomalies nécessitant une correction.

Cette démarche permet de conserver une trace structurée des vérifications réalisées avant la livraison de la version finale du système.

9.17 Conclusion

Ce chapitre a défini la stratégie de test et les principaux scénarios nécessaires à la validation du système de gestion interne de BMT Tourisme.

Les tests couvrent non seulement les fonctionnalités principales, mais également les règles métier, les permissions et le contrôle du périmètre multi-agences.

La validation finale de ces scénarios permettra de confirmer le comportement de l'application et d'identifier les éventuelles corrections nécessaires avant sa finalisation.

Le chapitre suivant sera consacré aux **difficultés rencontrées et aux solutions apportées** durant la réalisation du projet.

Chapitre 10

Difficultés Rencontrées et Solutions Apportées

10.1 Introduction

La réalisation d'une application de gestion comportant plusieurs modules, profils utilisateurs et règles métier nécessite de résoudre différentes problématiques techniques et fonctionnelles.

Durant le développement du système de gestion interne de BMT Tourisme, plusieurs difficultés ont été rencontrées, notamment au niveau de l'organisation du projet, de la gestion des autorisations, du fonctionnement multi-agences et de la cohérence entre les différents modules.

Ce chapitre présente les principales difficultés rencontrées ainsi que les solutions adoptées pour y répondre.

10.2 Structuration d'une application comportant plusieurs modules

L'une des premières difficultés concernait l'organisation générale du projet.

Le système devait couvrir plusieurs domaines fonctionnels : gestion des clients, dossiers, tâches, paiements, visas, billetterie, guides, missions, notifications et rapports.

Regrouper l'ensemble de ces fonctionnalités dans une seule structure aurait rendu le projet difficile à maintenir et à faire évoluer.

10.2.1 Solution apportée

Le projet a été organisé sous forme de plusieurs applications Django, chacune étant responsable d'un domaine fonctionnel précis.

Cette organisation a conduit à la séparation suivante :

— `accounts` pour les employés, rôles et permissions ;

— `agencies` pour les agences ;
— `clients` pour les clients ;
— `dossiers` pour les dossiers et documents ;
— `tasks` pour les tâches ;
— `payments` pour les opérations financières ;
— `visa` pour les demandes de visa ;
— `ticketing` pour la billetterie ;
— `guides` pour les guides et missions ;
— `notifications` pour les notifications et l'historique ;
— `reports` pour les rapports et indicateurs.

Cette architecture modulaire permet de mieux séparer les responsabilités et facilite les modifications futures.

10.3 Gestion flexible des rôles et permissions

Une autre difficulté importante concernait la gestion des accès.

Une approche reposant uniquement sur des rôles fixes aurait été trop restrictive. Deux employés ayant le même rôle peuvent, dans certaines situations, avoir besoin de droits différents.

Il était donc nécessaire de mettre en place un système plus flexible.

10.3.1 Solution apportée

La solution retenue repose sur trois éléments :

1. un profil **Employe** ;
2. un **Role** ;
3. un ensemble de **PermissionMetier**.

Les permissions peuvent être associées à un rôle, mais également directement à un employé.

Une fonction centralisée vérifie ensuite si l'utilisateur possède la permission nécessaire avant d'autoriser l'accès à une opération.

Cette solution permet de conserver des rôles métier clairs tout en autorisant des exceptions lorsque cela est nécessaire.

10.4 Protection réelle des fonctionnalités

Lors du développement d'une interface basée sur les permissions, il peut être tentant de simplement masquer les boutons et les éléments de navigation non autorisés.

Cette approche n'est cependant pas suffisante, car un utilisateur pourrait tenter d'accéder directement à une URL protégée.

10.4.1 Solution apportée

Les permissions ont été contrôlées directement au niveau des vues Django.

Un décorateur métier a été mis en place afin de vérifier l'autorisation avant l'exécution d'une fonctionnalité.

Ainsi, l'interface s'adapte aux permissions de l'utilisateur, mais la décision finale d'autoriser ou de refuser une opération reste effectuée côté serveur.

Cette séparation améliore la cohérence du contrôle d'accès.

10.5 Gestion du fonctionnement multi-agences

La prise en charge de plusieurs agences a constitué une difficulté importante dans la conception de l'application.

Un employé d'une agence ne doit pas automatiquement pouvoir consulter ou modifier les informations appartenant à une autre agence, même lorsqu'il possède la permission fonctionnelle correspondante.

Il était donc nécessaire de distinguer les permissions du périmètre organisationnel.

10.5.1 Solution apportée

Une logique de contrôle du périmètre a été mise en place.

Les utilisateurs disposant d'un accès global peuvent travailler sur un périmètre étendu, tandis que les autres employés sont limités aux données correspondant à leur agence d'affectation.

Les requêtes sont filtrées côté serveur avant la récupération ou la modification des objets.

Cette approche permet de distinguer clairement :

Permission fonctionnelle $\neq$ **Périmètre des données**

Un utilisateur peut donc avoir le droit de consulter un dossier sans pour autant disposer du droit de consulter tous les dossiers de toutes les agences.

10.6 Sécurisation des accès directs

Le filtrage des listes par agence ne suffit pas pour garantir la séparation des données.

Un utilisateur pourrait, par exemple, modifier manuellement l'identifiant présent dans une URL afin de tenter d'accéder à un objet appartenant à une autre agence.

10.6.1 Solution apportée

Le contrôle du périmètre est également appliqué lors de la récupération individuelle des objets.

Les pages de détail, de modification ainsi que les opérations sensibles vérifient que l'objet demandé appartient réellement au périmètre autorisé.

Le même principe est appliqué aux données reçues par les formulaires afin de limiter les tentatives de contournement par modification manuelle des requêtes.

10.7 Gestion des permissions des guides

La gestion des guides et des missions a nécessité une distinction entre la consultation d'une mission et son administration.

Un guide doit pouvoir consulter les missions qui le concernent et éventuellement mettre à jour certaines informations autorisées, sans nécessairement disposer du droit de créer ou d'administrer toutes les missions.

10.7.1 Solution apportée

Les droits ont été séparés en plusieurs permissions métier, notamment pour :

— consulter les missions ;
— gérer les missions ;
— modifier leur statut.

Cette séparation permet d'adapter le comportement du module aux responsabilités réelles du guide et des utilisateurs chargés de la planification.

10.8 Gestion de la règle métier liée à l'avance

La gestion des avances constitue une règle importante dans le fonctionnement du système.

Tous les dossiers ne doivent pas nécessairement être traités de la même manière. Un nouveau client peut nécessiter le versement d'une avance, tandis qu'un client autorisé peut bénéficier d'une exception.

Cette règle devait rester cohérente avec les paiements réellement enregistrés.

10.8.1 Solution apportée

Le dossier conserve l'information indiquant si une avance est requise.

Le client peut également disposer d'une autorisation lui permettant de poursuivre sans avance.

Le système détermine ensuite la possibilité de traitement en fonction de plusieurs conditions :

— le client est autorisé à poursuivre sans avance ;
— le dossier ne nécessite pas d'avance ;
— ou une avance a effectivement été enregistrée.

Cette logique permet d'éviter une règle rigide et de mieux représenter le fonctionnement métier.

10.9 Calcul des paiements et remboursements

La gestion financière a également nécessité une attention particulière.

Le montant réellement payé ne correspond pas toujours à une simple somme de toutes les opérations, car le système prend également en charge les remboursements.

10.9.1 Solution apportée

Les opérations financières sont enregistrées individuellement dans le modèle **Paiement**.

Le système distingue notamment :

— les avances ;

— les paiements ;
— les soldes ;
— les remboursements.

Le montant net payé est ensuite calculé en prenant en compte les opérations positives et les remboursements.

Le reste à payer est également calculé dynamiquement à partir du montant total du dossier.

Cette solution limite la duplication de valeurs calculables et réduit les risques d'incohérence.

10.10 Évolution des modèles et migrations

Au cours du développement, la structure des données a évolué afin de prendre en compte de nouveaux besoins.

L'ajout d'une relation obligatoire dans une table contenant déjà des données peut notamment poser un problème lors de la génération d'une migration.

10.10.1 Solution apportée

Certaines nouvelles relations ont été introduites progressivement.

Par exemple, lorsqu'une relation avec une agence devait être ajoutée à des données déjà existantes, elle pouvait être rendue temporairement facultative afin d'éviter d'imposer immédiatement une valeur artificielle aux anciens enregistrements.

Les données peuvent ensuite être régularisées avant de renforcer la contrainte si cela devient nécessaire.

Cette démarche permet de faire évoluer la structure de la base de manière progressive.

10.11 Connexion à la base de données pendant le développement

Le fonctionnement de Django dépend de la disponibilité du serveur MySQL utilisé par l'application.

Durant le développement, certaines commandes nécessitant un accès à la base peuvent échouer lorsque le service MySQL local n'est pas démarré.

10.11.1 Solution apportée

La disponibilité du service MySQL dans l'environnement WAMP doit être vérifiée avant l'exécution des opérations nécessitant une connexion à la base.

Cette distinction permet également d'éviter de confondre une erreur de connexion au serveur avec une erreur provenant des modèles ou des migrations Django.

10.12 Cohérence entre les différents modules

La présence de nombreux modules implique plusieurs dépendances.

Une tâche dépend d'un dossier, un paiement doit être rattaché au bon dossier, une mission dépend d'un guide et d'un dossier, et plusieurs opérations doivent respecter simultanément les permissions et le périmètre de l'agence.

La difficulté consiste donc à éviter qu'un module accepte des données incompatibles avec celles d'un autre module.

10.12.1 Solution apportée

Les contrôles métier et les règles de périmètre ont été intégrés dans les opérations concernées.

Les objets proposés dans les formulaires et ceux récupérés par les vues sont limités lorsque cela est nécessaire aux données autorisées.

Cette démarche permet de conserver une cohérence entre les différents composants du système.

10.13 Amélioration de l'interface utilisateur

Une première interface fonctionnelle permet de valider le fonctionnement général d'une application, mais elle ne répond pas nécessairement aux exigences de lisibilité et d'ergonomie attendues pour une utilisation quotidienne.

L'un des enjeux du projet consiste donc également à proposer une interface plus claire et plus professionnelle.

10.13.1 Solution apportée

Une identité visuelle commune a été définie pour les différentes pages de l'application.

Elle repose notamment sur :

— une navigation latérale structurée ;
— une barre supérieure commune ;
— des cartes pour les principaux indicateurs ;
— des tableaux homogènes ;
— des formulaires mieux organisés ;
— des badges permettant d'identifier les statuts ;
— une présentation adaptée aux différents modules ;
— une identité graphique cohérente avec BMT Tourisme.

L'interface reste commune aux différents profils, tandis que son contenu est adapté dynamiquement aux permissions disponibles.

10.14 Synthèse des difficultés

Le tableau suivant résume les principales difficultés rencontrées et les solutions adoptées.

Difficulté	Solution adoptée
Nombre important de fonctionnalités	Découpage du projet en applications Django spécialisées.
Gestion des différents profils	Association des rôles avec des permissions métier configurables.
Besoins particuliers d'un employé	Possibilité d'attribuer des permissions directement à l'employé.
Protection des fonctionnalités	Vérification des autorisations côté serveur.
Gestion multi-agences	Filtrage des données selon le périmètre de l'utilisateur.
Accès direct à des données non autorisées	Vérification du périmètre lors de la récupération des objets.
Gestion des guides	Séparation des permissions de consultation, gestion et changement de statut.
Règle d'avance	Calcul de l'autorisation de traitement selon le client, le dossier et les paiements.
Paiements et remboursements	Calcul dynamique des montants financiers.
Évolution de la base de données	Utilisation progressive des migrations Django.
Interface initialement simple	Mise en place d'une identité visuelle commune et d'interfaces plus structurées.

Table 10.1 – Synthèse des difficultés et solutions apportées

10.15 Apports de la résolution de ces difficultés

Les difficultés rencontrées ont permis d'améliorer progressivement la conception et l'architecture de l'application.

Elles ont notamment conduit à renforcer :

— la modularité du projet ;
— la séparation des responsabilités ;
— la gestion des autorisations ;
— la protection des données multi-agences ;
— la cohérence des règles métier ;
— la structuration de la base de données ;
— la maintenabilité du code ;
— l'ergonomie générale de l'application.

Ces améliorations contribuent à obtenir une solution mieux adaptée aux besoins définis pour le projet.

10.16 Conclusion

La réalisation du système de gestion interne de BMT Tourisme a nécessité de résoudre plusieurs difficultés liées aussi bien à la conception qu'à l'implémentation.

Les principales problématiques concernaient la modularité de l'application, la gestion des permissions, le contrôle multi-agences, les règles métier financières et la cohérence entre les différents modules.

Les solutions mises en place ont permis de structurer progressivement l'application et de renforcer son fonctionnement.

Le chapitre suivant présentera les **perspectives d'amélioration et les évolutions possibles** du système.

Chapitre 11

Perspectives d'Amélioration

11.1 Introduction

Le système de gestion interne développé pour BMT Tourisme répond aux principaux besoins fonctionnels identifiés dans le cadre de ce projet.

Cependant, une application de gestion est amenée à évoluer progressivement en fonction des besoins de l'entreprise, du nombre d'utilisateurs, du volume de données traité et des nouveaux services proposés.

Plusieurs améliorations peuvent ainsi être envisagées afin d'enrichir les fonctionnalités existantes, d'améliorer l'expérience utilisateur et de préparer le système à une utilisation plus étendue.

Ce chapitre présente les principales perspectives d'évolution du projet.

11.2 Déploiement sur un serveur de production

L'une des évolutions importantes consiste à déployer l'application dans un environnement de production accessible de manière sécurisée aux utilisateurs autorisés.

L'environnement local utilisé durant le développement pourra ainsi être remplacé par une infrastructure adaptée à une utilisation réelle.

Un tel déploiement nécessiterait notamment :

— un serveur d'application adapté ;
— un serveur de base de données configuré pour la production ;
— l'utilisation du protocole HTTPS ;
— une gestion sécurisée des paramètres sensibles ;
— une stratégie de sauvegarde des données ;
— une configuration adaptée aux performances et à la disponibilité du service.

11.3 Amélioration du système de notifications

Le système de notifications internes peut être enrichi afin d'améliorer la circulation de l'information entre les employés.

À terme, il serait possible de générer automatiquement des notifications lors de certains événements importants, par exemple :

— affectation d'une nouvelle tâche ;
— changement important dans un dossier ;
— attribution d'une mission à un guide ;
— approche d'une date d'échéance ;
— évolution du statut d'une demande de visa ;
— événement nécessitant l'intervention d'un employé.

Des notifications par courrier électronique pourraient également être envisagées pour certains événements importants.

11.4 Gestion avancée des documents

Le module documentaire peut être enrichi afin de faciliter davantage le traitement des dossiers.

Parmi les améliorations envisageables figurent :

— la classification des documents par catégorie ;
— la gestion de dates d'expiration ;
— la détection des documents manquants ;
— l'amélioration de la prévisualisation des fichiers ;
— la gestion de listes de documents requises selon le type de service ;
— l'archivage organisé des pièces associées aux dossiers.

Cette évolution serait particulièrement utile pour les dossiers nécessitant un nombre important de justificatifs.

11.5 Automatisation des tâches

Une évolution intéressante consisterait à automatiser la création de certaines tâches.

Lorsqu'un dossier atteint une étape déterminée, le système pourrait générer automatiquement les opérations nécessaires et les affecter au service concerné.

Par exemple, la création d'un dossier de visa pourrait entraîner la génération de tâches liées à la vérification des documents, au rendez-vous ou au suivi du résultat.

Cette automatisation permettrait de réduire certaines opérations manuelles et de standardiser davantage les processus internes.

11.6 Gestion avancée des échéances

Plusieurs éléments du système comportent des dates importantes, notamment les tâches, les visas, les billets et les missions.

Une évolution future pourrait permettre de mettre en place un système de suivi avancé des échéances.

L'application pourrait notamment :

— identifier les échéances proches ;
— signaler les tâches en retard ;
— afficher un calendrier des opérations ;
— générer des rappels ;
— mettre en évidence les éléments nécessitant une intervention rapide.

11.7 Tableaux de bord plus avancés

Le module de reporting peut être enrichi par de nouveaux indicateurs et des visualisations graphiques plus détaillées.

Les tableaux de bord pourraient notamment permettre d'analyser :

— l'évolution du nombre de dossiers ;
— la répartition des dossiers par service ;
— l'activité par agence ;
— les opérations financières sur une période donnée ;
— l'évolution des demandes de visa ;
— l'activité de billetterie ;
— les missions des guides ;
— la charge de travail des différents services.

Des filtres par période, agence ou type de service pourraient également faciliter l'analyse des informations.

11.8 Exportation des rapports

Une fonctionnalité d'export pourrait être ajoutée afin de permettre aux responsables d'exploiter les informations en dehors de l'application.

Les rapports pourraient notamment être exportés sous différents formats, tels que :

— PDF ;
— Excel ;
— CSV.

Cette fonctionnalité faciliterait l'archivage, le partage et l'analyse complémentaire des données.

11.9 Amélioration de la gestion financière

Le module financier peut évoluer vers une gestion plus complète des opérations réalisées par l'agence.

De futures versions pourraient intégrer :

— la génération de reçus ;
— la génération de factures ;
— une numérotation automatique des documents financiers ;
— un historique financier plus détaillé ;
— des rapports financiers par période ;
— des statistiques par agence ou par type de prestation.

Ces fonctionnalités permettraient de renforcer l'intégration entre le suivi des dossiers et la gestion financière.

11.10 Journalisation plus avancée

Le mécanisme d'historique existant peut être étendu afin d'obtenir une traçabilité encore plus détaillée.

Une évolution possible consisterait à conserver davantage d'informations sur certaines modifications importantes, notamment les changements de statut, les affectations, les modifications financières ou les opérations administratives.

Une interface de recherche et de filtrage de l'historique pourrait également être mise à disposition des utilisateurs autorisés.

11.11 Renforcement de la sécurité

Même si plusieurs mécanismes de contrôle d'accès sont intégrés dans l'application, la sécurité doit rester un processus d'amélioration continue.

Des évolutions pourraient notamment concerner :

— une politique renforcée de gestion des mots de passe ;
— l'authentification à deux facteurs ;
— la gestion avancée des sessions ;
— une journalisation plus détaillée des événements sensibles ;
— la mise en place de sauvegardes régulières ;
— le renforcement de la configuration de production.

11.12 Amélioration de l'expérience utilisateur

L'interface peut continuer à évoluer en fonction des retours des utilisateurs.

Les améliorations futures pourraient notamment porter sur :

— la simplification de certaines opérations fréquentes ;
— l'ajout de filtres de recherche avancés ;
— l'amélioration de l'affichage sur les petits écrans ;
— la personnalisation de certains éléments du tableau de bord ;
— l'amélioration des messages et confirmations ;
— l'ajout de raccourcis vers les actions fréquentes.

L'objectif serait de réduire le nombre d'étapes nécessaires pour effectuer les opérations quotidiennes.

11.13 Extension du fonctionnement multi-agences

L'architecture actuelle a été pensée afin de prendre en charge plusieurs agences.

Cette conception permet d'envisager l'intégration future de nouvelles agences sans remettre en cause l'organisation générale du système.

Des fonctionnalités supplémentaires pourraient également être développées pour faciliter la supervision globale, la comparaison des indicateurs entre agences et la gestion centralisée de certaines opérations.

11.14 Intégration avec des services externes

Une évolution plus avancée pourrait consister à connecter l'application à des services externes utilisés dans le secteur du tourisme.

Selon les besoins futurs de BMT Tourisme, des intégrations pourraient être étudiées avec des services de réservation, de communication ou d'autres plateformes métier.

Une telle évolution permettrait de limiter certaines saisies manuelles et d'améliorer la circulation des informations entre les différents systèmes.

11.15 Application mobile

À plus long terme, une interface mobile dédiée pourrait être envisagée pour certains profils.

Cette évolution pourrait être particulièrement utile pour les guides ou les employés travaillant régulièrement en dehors de l'agence.

Une application mobile pourrait notamment faciliter :

— la consultation des missions ;
— la réception des notifications ;
— la consultation de certaines informations de dossier ;
— la mise à jour de certains statuts autorisés.

11.16 Évolution vers une API

Une évolution de l'architecture pourrait également consister à mettre en place une API permettant à d'autres applications autorisées de communiquer avec le système.

Cette approche pourrait faciliter l'intégration future d'une application mobile ou de services externes sans modifier profondément la logique métier existante.

La mise en place d'une telle API nécessiterait toutefois une gestion rigoureuse de l'authentification, des permissions et de la sécurité des échanges.

11.17 Synthèse des perspectives

Les principales perspectives d'évolution peuvent être regroupées en plusieurs axes.

Axe	Évolutions possibles
Infrastructure	Déploiement en production, HTTPS et sauvegardes.
Communication	Notifications automatiques et notifications par e-mail.
Documents	Classification, expiration et suivi avancé des pièces.
Organisation	Automatisation des tâches et gestion avancée des échéances.
Décisionnel	Tableaux de bord avancés, filtres et nouveaux indicateurs.
Rapports	Export PDF, Excel et CSV.
Finance	Reçus, facturation et analyses financières complémentaires.
Sécurité	Authentification renforcée et journalisation avancée.
Multi-agences	Ajout de nouvelles agences et supervision centralisée.
Mobilité	Interface ou application mobile pour certains profils.
Interopérabilité	API et intégration future avec des services externes.

Table 11.1 – Synthèse des perspectives d'amélioration

11.18 Conclusion

Le système développé constitue une base permettant de centraliser plusieurs activités internes de BMT Tourisme au sein d'une même application.

Son architecture modulaire et la séparation entre les différents domaines fonctionnels permettent d'envisager plusieurs évolutions sans remettre en cause l'ensemble de la solution.

Les perspectives présentées dans ce chapitre concernent aussi bien l'amélioration des fonctionnalités existantes que l'ajout de nouveaux services, le renforcement de la sécurité, l'exploitation des données et l'ouverture du système vers d'autres plateformes.

Ces évolutions pourraient progressivement transformer la solution en un outil de gestion encore plus complet et mieux intégré à l'activité de BMT Tourisme.

Conclusion Générale

La transformation numérique des processus internes représente aujourd'hui un enjeu important pour les organisations souhaitant améliorer la gestion de leurs activités, la circulation de l'information et le suivi de leurs opérations.

C'est dans cette perspective que s'inscrit ce Projet de Fin d'Année, consacré à la conception et au développement d'un **système de gestion interne pour BMT Tourisme**.

L'objectif principal du projet était de proposer une solution centralisée permettant de structurer et de faciliter le suivi des différentes activités de l'agence. L'application développée regroupe ainsi plusieurs domaines métier au sein d'un même système, notamment la gestion des clients, des dossiers, des documents, des tâches, des paiements, des demandes de visa, de la billetterie, des guides et de leurs missions.

Une attention particulière a également été accordée à la gestion des utilisateurs. L'association entre les rôles et les permissions métier permet d'adapter les fonctionnalités accessibles aux responsabilités de chaque employé.

La dimension multi-agences a constitué un autre aspect important de la conception. Le système ne se limite pas à déterminer les opérations qu'un utilisateur peut effectuer, mais prend également en considération le périmètre des données sur lesquelles il est autorisé à intervenir.

Sur le plan technique, la solution repose principalement sur **Python**, le framework **Django** et une base de données relationnelle **MySQL**. L'utilisation de l'architecture MVT et l'organisation du projet en plusieurs applications spécialisées ont permis de séparer les différents domaines fonctionnels et de maintenir une structure claire.

La conception UML a permis de formaliser les interactions, les comportements et la structure générale du système avant leur traduction dans l'application. La modélisation des données a ensuite permis d'organiser les différentes entités et leurs relations autour des besoins métier identifiés.

La réalisation du projet a également nécessité la résolution de plusieurs problématiques, notamment la gestion flexible des permissions, la séparation des données entre les agences, la sécurisation des accès directs, la gestion de la règle d'avance ainsi que la cohérence des opérations financières.

Les solutions mises en place ont progressivement permis de construire une application modulaire dans laquelle les différentes fonctionnalités interagissent autour d'une structure commune.

Ce projet a également représenté une expérience enrichissante sur le plan technique et méthodologique. Il a permis de mettre en pratique plusieurs notions liées au développement web, à la modélisation UML, aux bases de données relationnelles, à l'architecture logicielle, à la sécurité des accès et à la traduction de besoins métier en fonctionnalités informatiques.

Le système réalisé constitue néanmoins une base pouvant continuer à évoluer. Plusieurs améliorations peuvent être envisagées, notamment le déploiement dans un environnement de production, l'automatisation de certaines tâches, l'amélioration des notifications, l'enrichissement des tableaux de bord, l'exportation des rapports, le renforcement de la sécurité et l'intégration future avec d'autres services.

En conclusion, ce projet a permis de concevoir une solution adaptée à la centralisation et au suivi des principales activités internes de BMT Tourisme, tout en mettant en place une architecture capable d'accompagner de futures évolutions fonctionnelles et techniques.

Bibliographie et Webographie

Les ressources suivantes ont été consultées ou constituent des références techniques relatives aux technologies, aux outils et aux méthodes utilisés dans le cadre de ce projet.

Documentation technique

1. **Django Software Foundation**, *Django Documentation*.
   Documentation officielle du framework Django, utilisée comme référence pour les modèles, les vues, les formulaires, l'authentification, les migrations et la gestion des données.
   https://docs.djangoproject.com/
2. **Python Software Foundation**, *Python Documentation*.
   Documentation officielle du langage Python.
   https://docs.python.org/3/
3. **Oracle Corporation**, *MySQL Documentation*.
   Documentation officielle du système de gestion de base de données MySQL.
   https://dev.mysql.com/doc/
4. **Bootstrap**, *Bootstrap Documentation*.
   Documentation officielle utilisée comme référence pour la conception des composants et des interfaces responsives.
   https://getbootstrap.com/docs/

Modélisation et conception

5. **Object Management Group (OMG)**, *Unified Modeling Language – UML*.
   Référence relative au langage de modélisation UML utilisé pour la conception des différents diagrammes du système.
   https://www.omg.org/spec/UML/

Ressources complémentaires

6. **Django Software Foundation**, *Django Authentication System*.
   Référence concernant les mécanismes d'authentification et de gestion des utilisa-

teurs proposés par Django.
https://docs.djangoproject.com/en/stable/topics/auth/

7. **Django Software Foundation**, *Django Models*.
   Référence concernant la définition des modèles, des relations et l'utilisation de l'ORM Django.
   https://docs.djangoproject.com/en/stable/topics/db/models/
8. **Django Software Foundation**, *Django Security*.
   Documentation relative aux mécanismes de sécurité disponibles dans Django.
   https://docs.djangoproject.com/en/stable/topics/security/

Dernière consultation : septembre 2026

Annexes

Les annexes regroupent des informations complémentaires relatives à l'organisation technique et au fonctionnement de l'application. Elles permettent d'apporter davantage de détails sans alourdir les chapitres principaux du rapport.

Annexe A – Organisation générale du projet

Le projet Django est organisé en plusieurs applications spécialisées, chacune étant responsable d'un domaine fonctionnel particulier.

```
ProjectBMT/
|
|-- ProjectBMT/
|
|-- accounts/
|-- agencies/
|-- clients/
|-- dossiers/
|-- tasks/
|-- payments/
|-- visa/
|-- ticketing/
|-- guides/
|-- notifications/
|-- reports/
|
|-- static/
|   |-- images/
|   |-- css/
|   `-- js/
|
|-- templates/
|
`-- manage.py
```

Cette organisation permet de séparer les responsabilités et de faciliter la maintenance

de l'application.

Annexe B – Principales entités du système

Le système repose principalement sur les entités suivantes :

Entité	Rôle principal
Agence	Représente une agence appartenant à l'organisation.
Employe	Représente le profil professionnel associé à un utilisateur.
Role	Définit un ensemble de responsabilités professionnelles.
PermissionMetier	Représente une autorisation fonctionnelle du système.
Client	Contient les informations relatives à un client.
Dossier	Centralise le traitement d'une demande client.
Document	Représente une pièce associée à un dossier.
Tache	Représente une opération à effectuer dans le cadre d'un dossier.
Paiement	Enregistre une opération financière associée à un dossier.
Visa	Permet de suivre une demande de visa.
Billet	Contient les informations relatives à une réservation ou à un billet.
Guide	Représente le profil spécifique d'un guide.
MissionGuide	Représente une mission affectée à un guide.
Notification	Permet de transmettre une information à un employé.
HistoriqueAction	Permet de conserver une trace de certaines opérations.

Table 2 – Principales entités de l'application

Annexe C – Permissions métier

La gestion des autorisations repose sur des permissions métier identifiées par des codes.

Les principales permissions définies dans le système sont présentées ci-dessous.

Code	**Description**
`clients.view`	Consulter les clients.
`clients.create`	Créer un client.
`clients.update`	Modifier un client.
`clients.delete`	Supprimer un client selon les règles définies.
`dossiers.view`	Consulter les dossiers.
`dossiers.create`	Créer un dossier.
`dossiers.update`	Modifier un dossier.
`documents.manage`	Gérer les documents associés aux dossiers.
`tasks.view`	Consulter les tâches.
`tasks.create`	Créer une tâche.
`tasks.update`	Modifier une tâche.
`tasks.assign`	Affecter une tâche à un employé.
`payments.view`	Consulter les opérations financières.
`payments.manage`	Gérer les paiements.
`visa.view`	Consulter les demandes de visa.
`visa.manage`	Gérer les demandes de visa.
`ticketing.view`	Consulter les informations de billetterie.
`ticketing.manage`	Gérer les billets et réservations.
`missions.view`	Consulter les missions.
`missions.manage`	Gérer les missions.

Code	**Description**
`missions.update_status`	Modifier le statut d'une mission selon les autorisations.
`employees.view`	Consulter les employés.
`employees.manage`	Gérer les employés.
`agencies.view`	Consulter les agences.
`agencies.manage`	Gérer les agences.
`reports.view`	Consulter les rapports et indicateurs.

Table 3 – Principales permissions métier

Annexe D – États des principaux éléments

Plusieurs entités utilisent des états permettant de suivre leur progression dans le système.

États d'un dossier

NOUVEAU → EN COURS → EN ATTENTE → TERMINE

Selon la situation, un dossier peut également être **BLOQUE** ou **ANNULE**.

États d'une tâche

A FAIRE → EN COURS → EN ATTENTE → TERMINEE

Une tâche peut également être **ANNULEE**.

États d'une demande de visa

DOCUMENTS MANQUANTS → DOCUMENTS COMPLETS → DOSSIER PRET → DEPOSE → EN TRAITEMENT

Le processus peut ensuite aboutir à :

ACCEPTE ou **REFUSE**

États d'un billet

Les principaux états utilisés pour la billetterie sont :

EN ATTENTE → RESERVE → BILLET EMIS

Un billet peut également être **ANNULE** ou **MODIFIE**.

États d'une mission

PLANIFIEE → EN COURS → TERMINEE

Une mission peut également être **ANNULEE**.

Annexe E – Principe du contrôle d'accès

Le contrôle d'accès repose sur plusieurs vérifications successives.

Utilisateur

↓

Authentification

↓

Profil Employé actif

↓

Rôle + Permissions métier

↓

Contrôle du périmètre de l'agence

↓

Validation de l'opération

Cette approche permet de distinguer l'autorisation fonctionnelle du périmètre des données accessibles.

Annexe F – Principe de la règle d'avance

Le traitement d'un dossier dépend notamment de la politique d'avance appliquée au client et au dossier.

Le principe général peut être résumé de la manière suivante :

Le client est-il autorisé sans avance ?

↓

Oui : traitement autorisé

Sinon

↓

Le dossier nécessite-t-il une avance ?

↓

Non : traitement autorisé

Sinon

↓

Une avance a-t-elle été versée ?

↓

Oui : traitement autorisé

Non : condition non satisfaite

Annexe G – Technologies principales

Technologie	Utilisation
Python	Langage principal utilisé pour la logique serveur.
Django	Framework utilisé pour le développement de l'application web.
MySQL	Système de gestion de base de données relationnelle.
HTML	Structure des interfaces web.
CSS	Mise en forme et personnalisation graphique.
Bootstrap	Composants et mise en page responsive.
JavaScript	Interactions dynamiques côté client.
WAMP	Environnement local utilisé notamment pour le service MySQL.
Git / GitHub	Gestion de versions et suivi du code source lorsque utilisé dans le cycle de développement.

Table 4 – Technologies principales du projet

Annexe H – Interfaces complémentaires

Cette partie pourra accueillir des captures d'écran supplémentaires qui ne sont pas indispensables dans le chapitre consacré à la réalisation, mais qui permettent d'illustrer plus largement le fonctionnement de l'application.

Les captures seront ajoutées après la finalisation complète de l'interface graphique.

Annexe I – Informations générales du projet

Projet	Système de Gestion Interne pour BMT Tourisme
Type de projet	Projet de Fin d'Année (PFA)
Réalisé par	Zakaria M'AYZOU
Encadrement académique	Mme Maryem El Hilali
Établissement	École Marocaine des Sciences de l'Ingénieur (EMSI)
Organisme d'accueil	BMT Tourisme
Année universitaire	2025–2026

Table 5 – Informations générales du projet

Conclusion des annexes

Ces annexes complètent les informations présentées dans le corps principal du rapport en regroupant plusieurs éléments techniques et organisationnels relatifs au système développé.

Elles permettent notamment de retrouver rapidement la structure du projet, les principales entités, les permissions métier, les états utilisés et les principes généraux de contrôle d'accès.